collaborative Protection Profile for Full Drive Encryption – Encryption Engine

Version: 3.0
2026-03-12
Full Disk Encryption international Technical Community

Revision History

Version	Date	Comment
0.1	2014-08-26	Initial release for iTC review
0.2	2014-09-05	Draft published for public review
0.13	2014-10-17	Incorporated comments received from the public review
1.0	2015-01-26	Incorporated comments received from the CCDB review
1.5	2015-09-02	Revised based on additional use cases developed by iTC
2.0	2016-09-09	Incorporated comments received from the public review, and also updated the Key Destruction section and AVA_VAN.
2.0 + Errata 20190201	2019-02-01	Updated to reflect CC Part 3 evaluation findings and FDE Interpretation Team [FIT] rulings
3.0	2026-03-12	Updated for CC:2022, reviewing and applying modifications to the cPP

Contents

1 Introduction

1.1 PP Overview

The purpose of the set of Collaborative Protection Profiles (cPPs) for Full Drive Encryption (FDE): Authorization Acquisition (AA) and Encryption Engine (EE) is to provide requirements for Data-at-Rest protection against unauthorized access or disclosure of stored data on a lost device. These cPPs allow FDE solutions based in software and/or hardware to meet the requirements for Data-at-Rest protection. The form factor for a storage device may vary, but could include: hard disk drives/solid state drives in servers, workstations, laptops, mobile devices, tablets, and external media. A hardware solution could be a Self-Encrypting Drive or other hardware-based solutions; the interface (USB, SATA, etc.) used to connect the storage device to the host machine is outside the scope of this cPP.

Full Drive Encryption encrypts all data (with certain exceptions) on the storage device and permits access to the data only after successful authorization to the FDE solution. The exceptions include the necessity to leave a portion of the storage device (the size may vary based on implementation) unencrypted for such things as the Master Boot Record (MBR) or other AA/EE pre-authentication software. These FDE cPPs interpret the term “full drive encryption” to allow FDE solutions to leave a portion of the storage device unencrypted so long as it does not contain plaintext user or plaintext authorization data.

Since the FDE cPPs support a variety of solutions, two cPPs describe the requirements for the FDE components shown in Figure 1.

Figure 1: FDE Components

The FDE cPP - Authorization Acquisition describes the requirements for the Authorization Acquisition piece and details the necessary security requirements and assurance activities necessary to interact with a user and result in the availability of a Border Encryption Value (BEV).

The FDE cPP - Encryption Engine describes the requirements for the Encryption Engine piece and details the necessary security requirements and assurance activities for the actual encryption and decryption of the data by the DEK. Each cPP will also have a set of core requirements for management functions, proper handling of cryptographic keys, updates performed in a trusted manner, audit and self-tests.

The Target of Evaluation (TOE) description defines the scope and functionality of the Encryption Engine, and the Security Problem Definition describes the assumptions made about the operating environment and the threats to the EE that the cPP requirements address.

1.2 Terms

The following sections list Common Criteria and technology terms used in this document.

1.2.1 Common Criteria Terms

Assurance	Grounds for confidence that a TOE meets the SFRs [CC].
Base Protection Profile (Base-PP)	Protection Profile used as a basis to build a PP-Configuration.
Collaborative Protection Profile (cPP)	A Protection Profile developed by international technical communities and approved by multiple schemes.
Common Criteria (CC)	Common Criteria for Information Technology Security Evaluation (International Standard ISO/IEC 15408).
Common Criteria Testing Laboratory	Within the context of the Common Criteria Evaluation and Validation Scheme (CCEVS), an IT security evaluation facility accredited by the National Voluntary Laboratory Accreditation Program (NVLAP) and approved by the NIAP Validation Body to conduct Common Criteria-based evaluations.
Common Evaluation Methodology (CEM)	Common Evaluation Methodology for Information Technology Security Evaluation.
Direct Rationale	A type of Protection Profile, PP-Module, or Security Target in which the security problem definition (SPD) elements are mapped directly to the SFRs and possibly to the security objectives for the operational environment. There are no security objectives for the TOE.
Extended Package (EP)	A deprecated document form for collecting SFRs that implement a particular protocol, technology, or functionality. See Functional Packages.
Functional Package (FP)	A document that collects SFRs for a particular protocol, technology, or functionality.
Operational Environment (OE)	Hardware and software that are outside the TOE boundary that support the TOE functionality and security policy.
Protection Profile (PP)	An implementation-independent set of security requirements for a category of products.
Protection Profile Configuration (PP-Configuration)	A comprehensive set of security requirements for a product type that consists of at least one Base-PP and at least one PP-Module.
Protection Profile Module (PP-Module)	An implementation-independent statement of security needs for a TOE type complementary to one or more Base-PPs.
Security Assurance Requirement (SAR)	A requirement to assure the security of the TOE.
Security Functional Requirement (SFR)	A requirement for security enforcement by the TOE.
Security Target (ST)	A set of implementation-dependent security requirements for a specific product.
Target of Evaluation (TOE)	The product under evaluation.
TOE Security Functionality (TSF)	The security functionality of the product under evaluation.

TOE Summary Specification (TSS)	A description of how a TOE satisfies the SFRs in an ST.

1.2.2 Technical Terms

Authorization Factor	A value that a user knows, has, or is (e.g. password, token, etc.) submitted to the TOE to establish that the user is in the community authorized to use the drive. This value is used in the derivation or decryption of the BEV and eventual decryption of the DEK. Note that these values may or may not be used to establish the particular identity of the user.
Authorized User	A user who has a valid Authorization Factor or legitimate physical possession of the drive.
Border Encryption Value (BEV)	A value passed from the FDE Authorization Acquisition (AA) to the FDE Encryption Engine (EE) intended to link the key chains of the two components.
Data Encryption Key (DEK)	A key used to encrypt data-at-rest.
Full Drive Encryption (FDE)	Refers to partitions of logical blocks of user accessible data as managed by the host system. FDE products encrypt all data (with certain exceptions) on the partition of the storage device and permits access to the data only after successful authorization. FDE solutions may leave a portion of the storage device unencrypted, for such things as the Master Boot Record (MBR) or other AA/EE pre-authentication software, so long as it contains no protected data.
Intermediate Key	A key used in a point between the initial user authorization and the DEK.
Key Chaining	The method of using multiple layers of encryption keys to protect data. A top layer key encrypts a lower layer key which encrypts the data; this method can have any number of layers.
Key Encryption Key (KEK)	A key used to encrypt other keys, such as DEKs or storage that contains keys.
Key Material	Key material is commonly known as critical security parameter (CSP) data, and also includes authorization data, nonces, and metadata.
Key Release Key (KRK)	A key used to release another key from storage, it is not used for the direct derivation or decryption of another key.
Key Sanitization	A method of sanitizing encrypted data by securely overwriting the key that was encrypting the data.
Non-Volatile Memory	A type of computer memory that will retain information without power.
Operating System (OS)	Software which runs at the highest privilege level and can directly control hardware resources.
Powered-Off State	The device has been shut down.
Protected Data	This refers to all encrypted data on the storage device. Protected data may exclude a Master Boot Record or Pre-authentication area of the drive – areas that are sometimes necessarily unencrypted.

Root of Trust for Update (RTU)	A type of RoT that verifies the integrity and authenticity of an update payload before initiating the update process.
Submask	A submask is a bit string that can be generated and stored in a number of ways. The individual SFRs provide context where these values may be used and what functions they perform, such as the BEV, intermediate key, or derived authentication value.

1.3 Implementation

Full Drive Encryption solutions vary with implementation and vendor combinations.

Vendors must evaluate products that provide both components of the Full Disk Encryption Solution (AA and EE) against both cPPs, although that could be done in a single evaluation with one ST. A vendor that provides a single component of an FDE solution would only evaluate against the applicable cPP. The FDE cPP is divided into two documents to allow labs to independently evaluate solutions tailored to one cPP or the other. When a customer acquires an FDE solution, they will either obtain a single vendor product that meets the AA + EE cPPs or two products, one of which meets the AA and the other of which meets the EE cPPs.

Table 1 illustrates a few examples for certification

Table 1: Examples of cPP Implementations

Implementation	cPP	Description
Host	AA	Host software provides the interface to a self-encrypting drive
Self-Encrypting Drive (SED)	EE	A self-encrypting drive used in combination with separate host software
Software FDE	AA + EE	A software full drive encryption solution
Hybrid	AA + EE	A single vendor’s combination of hardware (e.g., hardware encryption engine, cryptographic co-processor) and software / firmware

1.4 TOE Overview

The Target of Evaluation (TOE) for this cPP is either the Encryption Engine or a combined evaluation of the set of cPPs for FDE (Authorization Acquisition or Encryption Engine).

The following sections provide an overview of the functionality of the FDE EE as well as the security capabilities.

1.4.1 Encryption Engine Introduction

The Encryption Engine (EE) objectives focus on data encryption, policy enforcement, and key management. The EE is responsible for the generation, update, archival, recovery, protection, and destruction of the DEK and other intermediate keys under its control. The EE receives a Border Encryption Value (BEV) from the AA. The EE uses that BEV for the decryption of the DEK although other intermediate keys may exist in between those two points. Key Encryption Keys (KEKs) wrap other keys, notably the DEK or other intermediary keys which chain to the DEK. Key Releasing Keys (KRKs) authorize the EE to release either the DEK or other intermediary keys which chain to the DEK. These keys only differ in the functional use.

The EE determines whether to allow or deny a requested action based on the KEK or KRK provided by the AA. Possible requested actions include but are not limited to changing of encryption keys, decryption of data, and key sanitization of encryption keys (including the DEK). The EE may offer additional policy enforcement to prevent access to ciphertext or the unencrypted portion of the storage device. Additionally the EE may provide encryption support for multiple users on an individual basis.

Figure 2 illustrates the components within EE and its relationship with AA.

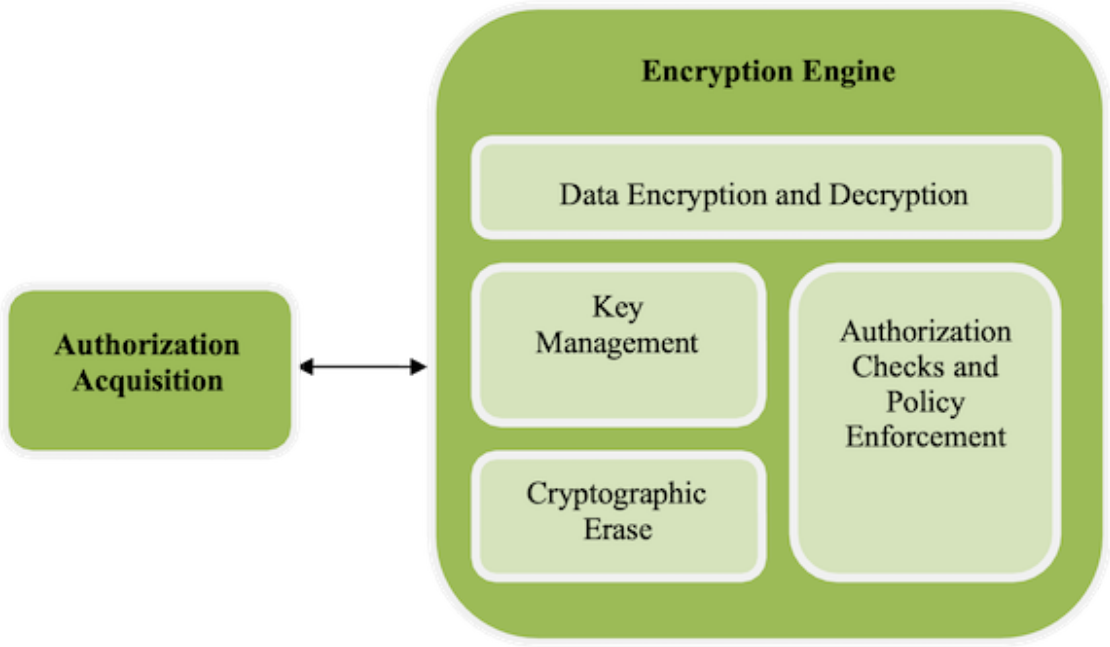


Figure 2: Encryption Engine Details

1.4.2 Encryption Engine Security Capabilities

The Encryption Engine is ultimately responsible for ensuring that the data is encrypted using a prescribed set of algorithms. The EE manages the decryption of the data on the storage device through decryption of the DEK based on the validity of the BEV provided by the AA. It also manages administrative functions, such as changing the DEK, managing the BEVs required for decrypting or releasing the DEK, managing the intermediate wrapping keys under its control, and performing a key sanitization.

The EE may provide key archiving and recovery functionality. The EE may manage the archiving and recovery itself, or interface with the AA to perform this function. It may also offer configurable features, which restricts the movement of keying material and disables recovery functionality.

The foremost security objective of encrypting storage devices is to force an adversary to perform an exhaustive search against a prohibitively large key space in order to recover the DEK or other intermediate keys. The EE uses approved cryptography to generate, handle, and protect keys to force an adversary who obtains an unpowered lost or stolen platform without the authorization factors or intermediate keys to exhaust the encryption key space of intermediate keys or DEK to obtain the data. The EE randomly generates DEKs and – in some cases - intermediate keys. The EE uses DEKs in a symmetric encryption algorithm in an appropriate mode along with appropriate initialization vectors for that mode to encrypt storage units (e.g. sectors or blocks) on the storage device. The EE either encrypts the DEK with a KEK or an intermediate key.

1.4.3 Interface/Boundary

The interface and boundary between the AA and the EE will vary based on the implementation. If one vendor provides the entire FDE solution, then it may choose to not implement an interface between the AA and EE components. If a vendor provides a solution for one of the components, then the assumptions below state that the channel between the

two components is sufficiently secure. Although standards and specifications exist for the interface between AA and EE components, the cPP does not require vendors to follow the standards in this version.

1.5 Compliant Targets of Evaluation

1.5.1 TOE Boundary

The environment in which the EE functions may differ depending on the boot stage of the platform in which it operates; see Figure 3. Aspects of initialization, and perhaps authorization may be performed in the Pre-Boot environment, while provisioning, encryption, decryption and management functionality are likely performed in the Operating System environment. Some of these aspects may occur in both environments. The Operating System environment may make a full range of services available to the Encryption Engine, including hardware drivers, cryptographic libraries, and perhaps other services external to the TOE. The Pre-Boot environment is much more constrained with limited capabilities. This environment turns on the minimum number of peripherals and loads only those drivers necessary to bring the platform from a cold start to executing a fully functional operating system with running applications. The EE TOE may include or leverage features and functions within the operational environment.

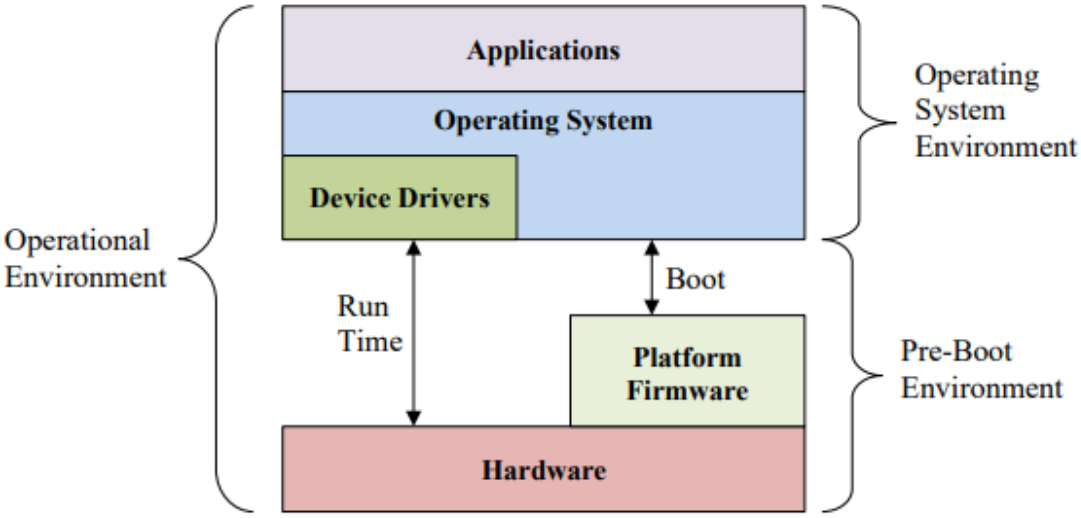


Figure 3: Operational Environment

1.6 Use Cases

The use case for a product conforming to the FDE cPPs is to protect data-at-rest on a device that is lost or stolen while powered off without any prior access by an adversary. The use case where an adversary obtains a device that is in a powered state and is able to make modifications to the environment or the TOE itself (e.g., evil maid attacks) is not addressed by these cPPs (i.e., FDE-AA and FDE- EE).

1.7 Product Features Mapped to Implementation-dependent Requirements

The features enumerated below, if implemented by the TOE, require that additional SFRs be claimed in the ST.

1.7.1 Symmetric Key Generation Support

A conformant TOE is required to implement proper generation of symmetric keys. This is selected based in FCS_CKM.1/DEK when the TSF generates a DEK. Or symmetric key generation may be used in key wrapping per FCS_COP.1/KeyWrap or key encryption per FCS_COP.1/KeyEnc.

If this feature is implemented by the TOE, the following requirements must be claimed in the ST:

- **FCS_CKM.1/SKG**

2 Conformance Claims

Conformance Statement

An ST must claim exact conformance to this PP.

The evaluation methods used for evaluating the TOE are a combination of the workunits defined in [CEM] as well as the Evaluation Activities for ensuring that individual SFRs and SARs have a sufficient level of supporting evidence in the Security Target and guidance documentation and have been sufficiently tested by the laboratory as part of completing ATE_IND.1. Any functional packages this PP claims similarly contain their own Evaluation Activities that are used in this same manner.

CC Conformance Claims

This PP is conformant to Part 2 (extended) and Part 3 (conformant) of Common Criteria CC:2022, Revision 1.

PP Claim

This PP does not claim conformance to any Protection Profile.

There are no PPs or PP-Modules that are allowed in a PP-Configuration with this PP.

Package Claim

This PP is not conformant to any Functional or Assurance Packages.

3 Security Problem Definition

3.1 Threats

This section provides a narrative that describes how the requirements mitigate the mapped threats. A requirement may mitigate aspects of multiple threats. A requirement may only mitigate a threat in a limited way. Some requirements are optional, either because the TSF fully mitigates the threat without the additional requirements being claimed or because the TSF relies on its Operational Environment to provide the functionality that is described by the optional requirements.

A threat consists of a threat agent, an asset and an adverse action of that threat agent on that asset. The threat agents are the entities that put the assets at risk if an adversary obtains a lost or stolen storage device. Threats drive the functional requirements for the Target of Evaluation (TOE). For instance, one threat below is T.UNAUTHORIZED_DATA_ACCESS. The threat agent is the possessor (unauthorized user) of a lost or stolen storage device. The asset is the data on the storage device, while the adverse action is to attempt to obtain those data from the storage device. This threat drives the functional requirements for the storage device encryption (TOE) to authorize who can use the TOE to access the hard disk and encrypt/decrypt the data. Since possession of the KEK, DEK, intermediate keys, authorization factors, submasks, and random numbers or any other values that contribute to the creation of keys or authorization factors could allow an unauthorized user to defeat the encryption, this SPD considers key material equivalent to the data in importance and they appear among the other assets addressed below.

It is important to reemphasize at this point that this collaborative Protection Profile does not expect the product (TOE) to defend against the possessor of the lost or stolen storage device who can introduce malicious code or exploitable hardware components into the Target of Evaluation (TOE) or the Operational Environment. It assumes that the user physically protects the TOE and that the Operational Environment provides sufficient protection against logical attacks. One specific area where a conformant TOE offers some protection is in providing updates to the TOE; other than this area, though, this cPP mandates no other countermeasures. Similarly, these requirements do not address the “lost and found” hard disk problem, where an adversary may have taken the hard disk, compromised the unencrypted portions of the boot device (e.g., MBR, boot partition), and then made it available to be recovered by the original user so that they would execute the compromised code.

T.AUTHORIZATION_GUESSING

Threat agents may exercise host software to repeatedly guess authorization factors, such as passwords and PINs. Successful guessing of the authorization factors may cause the TOE to release DEKs or otherwise put it in a state in which it discloses protected data to unauthorized users.

T.CHOSEN_PLAINTEXT

Threat agents may trick authorized users into storing chosen plaintext on the encrypted storage device in the form of an image, document, or some other file. A poor choice of encryption algorithms, encryption modes, and initialization vectors along with the chosen plaintext could allow attackers to recover the effective DEK, thus providing unauthorized access to the previously unknown plaintext on the storage device.

T.KEYING_MATERIAL_COMPROMISE

Possession of any of the keys, authorization factors, submasks, and random numbers or any other values that contribute to the creation of keys or authorization factors could allow an unauthorized user to defeat the encryption. The cPP considers possession of key material of equal importance to the data itself. Threat agents may look for keying material in unencrypted sectors of the storage device and on other peripherals in the operational environment (OE), (e.g., BIOS configuration, SPI flash, or TPMs).

T.KEYSPACE_EXHAUST

Threat agents may perform a cryptographic exhaustive search against the key space. Poorly chosen encryption algorithms and parameters allow attackers to exhaust the key space through brute force and give them unauthorized access to the data.

T.KNOWN_PLAINTEXT

Threat agents know plaintext in regions of storage devices, especially in uninitialized regions (all zeroes) as well as regions that contain well known software such as operating systems. A poor choice of encryption algorithms, encryption modes, and initialization vectors along with known plaintext could allow an attacker to recover the effective DEK, thus providing unauthorized access to the previously unknown plaintext on the storage device.

T.UNAUTHORIZED_DATA_ACCESS

The cPP addresses the primary threat of unauthorized disclosure of protected data stored on a storage device. If an adversary obtains a lost or stolen storage device (e.g., a storage device contained in a laptop or a portable external storage device), they may attempt to connect a targeted storage device to a host of which they have complete control and have raw access to the storage device (e.g., to specified disk sectors, to specified blocks).

T.UNAUTHORIZED_FIRMWARE_MODIFY

An attacker attempts to modify the firmware of the storage device via a command from the AA or from the OE that may compromise the security features of the TOE.

T.UNAUTHORIZED_UPDATE

Threat agents may attempt to perform an update of the product which compromises the security features of the TOE. Poorly chosen update protocols, signature generation and verification algorithms, and parameters may allow attackers to install software that bypasses the intended security features and provides them unauthorized access to data.

3.2 Assumptions

A.INITIAL_DRIVE_STATE

Users enable Full Drive Encryption on a newly provisioned storage device free of protected data in areas not targeted for encryption. It is also assumed that data intended for protection should not be on the targeted storage media until after provisioning. The cPP does not intend to include requirements to find all the areas on storage devices that potentially contain protected data. In some cases, it may not be possible - for example, data contained in “bad” sectors. While inadvertent exposure to data contained in bad sectors or un-partitioned space is unlikely, one may use forensics tools to recover data from such areas of the storage device. Consequently, the cPP assumes bad sectors, un-partitioned space, and areas that must contain unencrypted code (e.g., MBR and AA/EE pre-authentication software) contain no protected data.

A.PHYSICAL

The platform is assumed to be physically protected in its Operational Environment and not subject to physical attacks that compromise the security or interfere with the platform’s correct operation.

A.PLATFORM_STATE

The platform in which the storage device resides (or an external storage device is connected) is free of malware that could interfere with the correct operation of the product.

A.POWER_DOWN

The user does not leave the platform and/or storage device unattended until all volatile memory is erased after a power-off, so memory remnant attacks are infeasible.

Authorized users do not leave the platform and/or storage device in a mode where sensitive information persists in non-volatile storage (e.g., lock screen). Users power the platform and/or storage device down or place it into a power managed state, such as a “hibernation mode”.

A.STRONG_CRYPTO

All cryptography implemented in the Operational Environment and used by the TOE meets the requirements listed in the cPP. This includes generation of external token authorization factors by an RBG.

A.TRAINED_USER

Users follow the provided guidance for securing the TOE and authorization factors. This includes conformance with authorization factor strength, using external token authentication factors for no other purpose and ensuring external token authorization factors are securely stored separately from the storage device or platform. The user should also be trained on how to power off their system.

A.TRUSTED_CHANNEL

Communication among and between product components (e.g., AA and EE) is sufficiently protected to prevent information disclosure. In cases in which a single product fulfils both cPPs, the communication between the components does not extend beyond the boundary of the TOE (i.e., communication path is within the TOE boundary) so this assumption is inherently met. In cases in which independent products satisfy the requirements of the AA and EE, the physically close proximity of the two products during their operation means that the threat agent has very little opportunity to interpose itself in the channel between the two without the user noticing and taking appropriate actions.

3.3 Organizational Security Policies

This PP defines no Organizational Security Policies.

4 Security Objectives

4.1 Security Objectives for the Operational Environment

The Operational Environment (OE) of the TOE implements technical and procedural measures to assist the TOE in correctly providing its security functionality. This part wise solution forms the security objectives for the Operational Environment and consists of a set of statements describing the goals that the Operational Environment should achieve.

OE.INITIAL_DRIVE_STATE

The OE provides a newly provisioned or initialized storage device free of protected data in areas not targeted for encryption.

Rationale: Since the cPP requires all protected data to be encrypted, A.INITIAL_DRIVE_STATE assumes that the initial state of the device targeted FDE is free of protected data in those areas of the drive where encryption will not be invoked (e.g., MBR, AA, or EE pre-authentication software). Given this known start state, the product (once installed and operational) ensures partitions of logical blocks of user accessible data is protected.

OE.PASSWORD_STRENGTH

An authorized user will be responsible for ensuring that the password authorization factor conforms to guidance from the organization that uses or owns the TOE.

Rationale: Users are properly trained [A.TRAINED_USER] to create authorization factors that conform to administrative guidance.

OE.PHYSICAL

The Operational Environment will provide a secure physical computing space such than an adversary is not able to make modifications to the environment or to the TOE itself.

Rationale: As stated in section 1.6, the use case for this cPP is to protect data-at-rest on a device where the adversary receives it in a powered off state and has no prior access.

OE.PLATFORM_STATE

The platform in which the storage device resides (or an external storage device is connected) is free of malware that could interfere with the correct operation of the product.

Rationale: A platform free of malware [A.PLATFORM_STATE] prevents an attack vector that could potentially interfere with the correct operation of the product.

OE.POWER_DOWN

Volatile memory is erased after entering a compliant power-saving state or turned off so memory remnant attacks are infeasible.

Rationale: Users are properly trained [A.TRAINED_USER] to not leave the storage device unattended until it is in a compliant power-saving state or fully turned off.

OE.SINGLE_USE_ET

External tokens that contain authorization factors will be used for no other purpose than to store the external token authorization factor.

Rationale: Users are properly trained [A.TRAINED_USER] to use external token authorization factors as intended and for no other purpose.

OE.STRONG_ENVIRONMENT_CRYPTO

The Operational Environment will provide a cryptographic function capability that is commensurate with the requirements and capabilities of the TOE and Appendix A.

Rationale: All cryptography implemented in the Operational Environment and used by the product meets the requirements listed in this cPP [A.STRONG_CRYPTO].

OE.TRAINED_USERS

Authorized users will be properly trained and follow all guidance for securing the TOE and authorization factors.

Rationale: Users are properly trained [A.TRAINED_USER] to create authorization factors that conform to guidance, not store external token authorization factors with the device, and power down the TOE when required [OE.PLATFORM_STATE].

OE.TRUSTED_CHANNEL

Communication among and between product components (i.e., AA and EE) is sufficiently protected to prevent information disclosure.

Rationale: In situations where there is an opportunity for an adversary to interpose themselves in the channel between the AA and the EE, a trusted channel should be established to prevent exploitation. [A.TRUSTED_CHANNEL] assumes the existence of a trusted channel between the AA and EE, except for when the boundary is within and does not breach the TOE or is in such close proximity that a breach is not possible without detection.

4.2 Security Objectives Rationale

This section describes how the assumptions and organizational security policies map to operational environment security objectives.

Table 2: Security Objectives Rationale

Assumption or OSP	Security Objectives	Rationale
A.INITIAL_DRIVE_STATE	OE.INITIAL_DRIVE_STATE	The operational environment objective OE.INITIAL_DRIVE_STATE is realized through A.INITIAL_DRIVE_STATE.
A.PHYSICAL	OE.PHYSICAL	The operational environment objective OE.PHYSICAL is realized through A.PHYSICAL.
A.PLATFORM_STATE	OE.PLATFORM_STATE	The operational environment objective OE.PLATFORM_STATE is realized through A.PLATFORM_STATE.
A.POWER_DOWN	OE.POWER_DOWN	The operational environment objective OE.POWER_DOWN is realized through A.POWER_DOWN.
A.STRONG_CRYPTO	OE.STRONG_ENVIRONMENT_CRYPTO	The operational environment objective OE.STRONG_ENVIRONMENT_CRYPTO is realized through A.STRONG_CRYPTO.
A.TRAINED_USER	OE.PASSWORD_STRENGTH	The operational environment objective OE.PASSWORD_STRENGTH is realized through A.TRAINED_USER.

	OE.POWER_DOWN	The operational environment objective OE.POWER_DOWN is realized through A.TRAINED_USER.
	OE.SINGLE_USE_ET	The operational environment objective OE.SINGLE_USE_ET is realized through A.TRAINED_USER.
	OE.TRAINED_USERS	The operational environment objective OE.TRAINED_USERS is realized through A.TRAINED_USER.
A.TRUSTED_ CHANNEL	OE.TRUSTED_ CHANNEL	The operational environment objective OE.TRUSTED_CHANNEL is realized through A.TRUSTED_CHANNEL.

5 Security Requirements

This chapter describes the security requirements which have to be fulfilled by the product under evaluation. Those requirements comprise functional components from Part 2 and assurance components from Part 3 of [CC]. The following conventions are used for the completion of operations:

- **Refinement** operation (denoted by **bold text** or ~~strikethrough text~~): Is used to add details to a requirement or to remove part of the requirement that is made irrelevant through the completion of another operation, and thus further restricts a requirement.
- **Selection** (denoted by *italicized text*): Is used to select one or more options provided by the [CC] in stating a requirement.
- **Assignment** operation (denoted by *italicized text*): Is used to assign a specific value to an unspecified parameter, such as the length of a password. Showing the value in square brackets indicates assignment.
- **Iteration** operation: Is indicated by appending the SFR name with a slash and unique identifier suggesting the purpose of the operation, e.g. "/EXAMPLE1."

5.1 Security Functional Requirements

The individual security functional requirements are specified in the sections below.

5.1.1 Cryptographic Support (FCS)

FCS_CKM.1/DEK Cryptographic Key Generation (Data Encryption Key)

FCS_CKM.1.1/DEK

The TSF shall generate cryptographic keys in accordance with a specified cryptographic key generation ~~algorithm~~ **method** [**selection**: *generate a DEK using the RBG as specified in FCS_CKM.1/SKG, accept a DEK that is generated by the RBG provided by the OE, accept a DEK that is wrapped as specified in FCS_COP.1/KeyWrap*] and specified cryptographic key sizes [256 bits] ~~that meet the following: [~~**assignment**: *list of standards*~~]~~.

Application Note: This SFR is iterated because additional iterations are defined as optional requirements in Appendix A. This iteration was chosen specifically to ensure consistency between the FDE cPPs.

The purpose of this requirement is to explain DEK generation during provisioning.

If the TOE can be configured to obtain a DEK through more than one method, the ST author chooses the applicable options within the selection. For example, the TOE may generate random numbers with an approved RBG to create a DEK, as well as provide an interface to accept a DEK from the environment.

If the ST author chooses the first or third option, or both in the selection, the corresponding requirement is pulled from Appendix A and included in the body of the ST.

FCS_CKM.6/Power Cryptographic Key and Key Material Destruction (Power Management)

FCS_CKM.6.1/Power

The TSF shall destroy [*all key material, BEV, and authentication factors stored in plaintext*] when [*transitioning to a compliant power saving state as defined by FPT_PWR_EXT.1*].

Application Note: The TOE may end up in a non-compliant power saving state indistinguishable from a compliant power state (e.g. as result of sudden or unexpected power loss). Guidance documentation must state what conditions result in clear text keys or key materials to stay in volatile memory and identify mitigation measures that result in erasure of volatile memory.

FCS_CKM.6.2/Power

The TSF shall destroy cryptographic keys and keying material specified by FCS_CKM.6.1/Power in accordance with a specified cryptographic key destruction method **[selection: *instruct the operational environment to erase*, *erase*]** that meets the following: [*a key destruction method specified in FCS_CKM_EXT.6*].

Application Note: In some cases, erasure of keys from volatile memory is only supported by the operational environment, in which case the operational environment must expose a well-documented mechanism or interface to invoke the memory erase operation.

Self-encrypting drives do not store keys in the Operational Environment and cannot instruct the Operational Environment to perform functionality so they are not expected to select “instruct the Operational Environment to clear”.

FCS_CKM_EXT.6 Cryptographic Key Destruction Types

FCS_CKM_EXT.6.1

The TSF shall use [**selection**: *FCS_CKM.6/GENHW*, *FCS_CKM.6/SW*, *FCS_CKM.6/TOEHW*] and [**selection**: *FCS_CKM.6/KEK*, *no other method*] key destruction methods.

Application Note: The specific key destruction methods that will be claimed are dependent on the selections made for where keys are stored.

If multiple selections are made, the TSS should identify which keys are destroyed according to which selections.

FCS_COP.1/Hash Cryptographic Operation (Hash Algorithm)

FCS_COP.1.1/Hash

The TSF shall perform [*cryptographic hashing*] in accordance with a specified cryptographic algorithm [**selection**: *SHA-256, SHA-384, SHA-512, SHA3-384, SHA3-512*] that meet the following: [**selection**: *ISO/IEC 10118-3:2018 [SHA, SHA3], FIPS PUB 180-4 [SHA], FIPS PUB 202 [SHA3]*].

Application Note:

- SHA3 hashes may be used for internal hardware functionality such as boot integrity checks and DRBGs, and
- SHA-256 is permitted only for use as a PRF, including as part of a key derivation function or RBG, or as part of LMS or XMSS.

The hash selection should be consistent with the overall strength of the algorithm used for signature generation. For example, the TOE should choose SHA-384 for 3072-bit RSA, 4096-bit RSA, or ECC with P-384; and SHA-512 for ECC with P-521.

FCS_COP.1/SigVer Cryptographic Operation - Signature Verification

FCS_COP.1.1/SigVer

The TSF shall perform [digital signature verification] in accordance with a specified cryptographic algorithm [**selection**: *Cryptographic algorithm*] and cryptographic key sizes [**selection**: *Cryptographic key sizes*] that meet the following: [**selection**: *List of standards*]

Table 3 provides the allowable choices for completion of the selection operations of FCS_COP.1/SigVer.

Table 3: Allowable choices for FCS_COP.1/SigVer

Identifier	Cryptographic algorithm	Cryptographic key sizes	List of standards
RSA-PKCS	RSASSA-PKCS1-v1_5	Modulus of size [**selection**: *3072*, *4096*, *6144*, *8192*] bits and hash[**selection**: *SHA-384*, *SHA-512*]	RFC 8017 (Section 8.2) [PKCS #1 v2.2] FIPS PUB 186-5 (Section 5.4) [RSASSA-PKCS1-v1_5]
RSA-PSS	RSASSA-PSS	Modulus of size [**selection**: *3072*, *4096*, *6144*, *8192*] bits and hash[**selection**: *SHA-384*, *SHA-512*]	RFC 8017 (Section 8.1) [PKCS#1 v2.2] FIPS PUB 186-5 (Section 5.4) [RSASSA-PSS]
ECDSA	ECDSA	Elliptic Curve [**selection**: *P-384*, *P-521*] using hash [**selection**: *SHA-384*, *SHA-512*]	[**selection**: *ISO/IEC 14888-3:2018 (Subclause 6.6)*, *FIPS PUB 186-5 (Section 6.4.2)*] [ECDSA] NIST SP-800 186 (Section 4) [NIST Curves]
LMS	LMS	Private key size = [**selection**: • *192 bits with [**selection**: SHA-256/192, SHAKE256/192]* • *256 bits with [**selection**: SHA-256, SHAKE256]*] Winternitz parameter = [**selection**: *1*, *2*, *4*, *8*] Tree height = [**selection**: *5*, *10*, *15*, *20*, *25*]	RFC 8554 [LMS] NIST SP 800-208 [parameters]
XMSS	XMSS	Private key size = [**selection**:	RFC 8391 [XMSS]

		• *192 bits with [**selection**: SHA-256/192, SHAKE256/192]* • *256 bits with [**selection**: SHA-256, SHAKE256]*] Tree height = [**selection**: *10*, *16*, *20*]	NIST SP 800-208 [parameters]
ML-DSA	ML-DSA Signature Verification	Parameter set = ML-DSA-87	NIST FIPS 204 (Section 5.3)

Application Note: The ST Author should choose the algorithm implemented to perform verification of digital signatures. For the algorithm chosen, the ST Author should make the appropriate assignments/selections to specify the parameters that are implemented for that algorithm. In particular, if ECDSA is selected as one of the signature algorithms, the key size specified must match the selection for the curve used in the algorithm.

For elliptic curve-based schemes, the key size refers to the binary logarithm (log2) of the order of the base point.

FCS_COP.1/SKC Cryptographic Operation (AES Data Encryption/Decryption)

FCS_COP.1.1/SKC

The TSF shall perform [symmetric-key data encryption/decryption] in accordance with a specified cryptographic algorithm [**selection**: *Cryptographic algorithm*] and cryptographic key sizes [**selection**: *Cryptographic key sizes*] that meet the following: [**selection**: *List of standards*]

Table 4 provides the allowed choices for completion of the selection operations of FCS_COP.1/SKC.

Table 4: Allowed choices for FCS_COP.1/SKC

Identifier	Cryptographic algorithm	Cryptographic key sizes	List of standards
AES-CBC	AES in CBC mode with non-repeating and unpredictable IVs	256 bits	[**selection**: *ISO/IEC 18033-3:2010 (Subclause 5.2), FIPS PUB 197*] [AES] [**selection**: *ISO/IEC 10116:2017 (Clause 7), NIST SP 800-38A*] [CBC]
AES-XTS	AES in XTS mode with unique tweak values that are	512 bits	[**selection**: *ISO/IEC 18033-*

	consecutive non-negative integers starting at an arbitrary non-negative integer		*3:2010 (Subclause 5.2), FIPS PUB 197*] [AES] [**selection**: *IEEE Std. 1619-2018, NIST SP 800-38E*] [XTS]
AES-GCM	AES in GCM mode with non-repeating IVs using [**selection**: *deterministic*, *RBG-based*], IV construction; the tag must be of length [**selection**: *96*, *104*, *112*, *120*, *128*] bits.	256 bits	[**selection**: *ISO/IEC 18033-3:2010 (Subclause 5.2), FIPS PUB 197*] [AES] [**selection**: *ISO/IEC 19772:2020 (Clause 10), NIST SP 800-38D*] [GCM]

Application Note: The intent of this requirement in the context of this cPP is to provide an SFR that expresses the appropriate symmetric encryption/decryption algorithms suitable for use in the TOE. If the ST author incorporates the validation requirement (FCS_VAL_EXT.1) and chooses to select the option to decrypt a known value and perform a comparison, this is the requirement used to specify the algorithm, modes, and key sizes the ST author can choose from.

FCS_KYC_EXT.2 Key Chaining (Recipient)

FCS_KYC_EXT.2.1

The TSF shall accept a BEV of at least 256 bits.

FCS_KYC_EXT.2.2

The TSF shall maintain a chain of intermediary keys originating from the BEV to the DEK using the following methods: [**selection**:

- *key encryption as specified in FCS_COP.1/KeyEnc*
- *key transport as specified in FCS_COP.1/KeyEncap*
- *key wrapping as specified in FCS_COP.1/KeyWrap*
- *key derivation as specified in FCS_CKM.5*
- *key combining as specified in FCS_SMC_EXT.1*

] while maintaining an effective strength of [256 bits] for symmetric keys and an effective strength of [**selection**: *not applicable*, *128 bits*, *192 bits*, *256 bits*] for asymmetric keys.

Application Note: Key Chaining is the method of using multiple layers of encryption keys to ultimately secure the protected data encrypted on the drive. The number of intermediate keys will vary – from two (e.g., using the BEV as an intermediary key to wrap the DEK) to many. This applies to all keys that contribute to the ultimate wrapping or derivation of the DEK; including those in areas of protected storage (e.g. TPM stored keys, comparison values).

The BEV is considered to be equivalent to keying material and therefore additional checksums or similar values are not the BEV, even if they are sent with the BEV.

Once the ST author has selected a method to create the chain (either by deriving keys or unwrapping them), they pull the appropriate requirement out of Appendix B. It is allowable for an implementation to use both methods.

The method the TOE uses to chain keys and manage/protect them is described in the Key Management Description; see Appendix E for more information.

FCS_SNI_EXT.1 Cryptographic Operation (Salt, Nonce, and Initialization Vector Generation)

FCS_SNI_EXT.1.1

The TSF shall [**selection**:

- *use no salts*
- *use salts that are generated by a [**selection**: DRBG as specified in FCS_RBG.1, DRBG provided by the OE]*

].

FCS_SNI_EXT.1.2

The TSF shall use [**selection**: *no nonces*, *unique nonces with a minimum size of [64] bits*].

FCS_SNI_EXT.1.3

The TSF shall [**selection**:

- *use no IVs*
- *create IVs in the following manner [**selection**:*
  - *CBC: IVs shall be non-repeating and unpredictable*
  - *CCM: Nonce shall be non-repeating and unpredictable*
  - *XTS: No IV. Tweak values shall be non-negative integers, assigned consecutively, and starting at an arbitrary non-negative integer;*
  - *GCM: IV shall be non-repeating. The number of invocations of GCM shall not exceed 2^32 for a given secret key*

  ]

].

Application Note: This requirement covers several important factors – the salt must be random, but the nonces only have to be unique. FCS_SNI_EXT.1.3 specifies how the IV should be handled for each encryption mode. CBC, XTS, and GCM are allowed for AES encryption of the data. AES-CCM is an allowed mode for Key Wrapping.

If the TSF uses salts in support of cryptographic operations, and these salts are generated by the TSF, then FCS_RBG.1 must be claimed.

FCS_VAL_EXT.1 Validation

FCS_VAL_EXT.1.1

The TSF shall perform validation of the [*BEV*] using the following methods: [**selection**:

- *key wrap as specified in FCS_COP.1/KeyWrap;*
- *hash the [BEV] as specified in [**selection**: FCS_COP.1/Hash, FCS_COP.1/KeyedHash] and compare it to a stored hashed [**value**];*
- *decrypt a known value using the [**selection**: intermediate key, BEV] specified in FCS_COP.1/SKC and compare it against a stored known value*

].

FCS_VAL_EXT.1.2

The TSF shall require validation of the [*BEV*] prior to [*allowing access to TSF data after exiting a Compliant power saving state*].

Application Note: This SFR is claimed when the TSF validates an authentication factor as a prerequisite to unlocking the key chain as defined in FCS_KYC_EXT.2.

FCS_VAL_EXT.1.3

The TSF shall [**selection**:

- *perform a key sanitization of the DEK upon a [**selection**: configurable number, [**assignment**: ST author specified number]] of consecutive failed validation attempts*
- *institute a delay such that only [**assignment**: ST author specified number of attempts] can be made within a 24 hour period*
- *block validation after [**assignment**: ST author specified number of attempts] of consecutive failed validation attempts*
- *require power cycle or reset of the TOE after [**assignment**: ST author specified number of attempts] of consecutive failed validation attempts*

].

Application Note: "Validation" of the BEV can occur at any point in the key chain, including when the DEK is decrypted. For the purposes of this requirement, validating a key derived from the BEV equates to “validating” the BEV. The purpose of performing secure validation is to not expose any material that might compromise the submask(s).

The TOE validates the BEV prior to processing (e.g. decryption, derivation) of the keychain. When the key wrap in FCS_COP.1/KeyWrap is used, the validation is performed inherently.

The delay must be enforced by the TOE, but this requirement is not intended to address attacks that bypass the product (e.g. attacker obtains hash value or “known” crypto value and mounts attacks outside of the TOE, such as a third party password crackers). The cryptographic functions (i.e., hash, decryption) performed are those specified in FCS_COP.1/Hash, FCS_COP.1/KeyedHash, and FCS_COP.1/SKC.

5.1.2 User Data Protection

FDP_DSK_EXT.1 Protection of Data on Disk

FDP_DSK_EXT.1.1

The TSF shall perform full drive encryption in accordance with FCS_COP.1/SKC, such that the drive contains no plaintext protected data.

FDP_DSK_EXT.1.2

The TSF shall encrypt all protected data without user intervention.

Application Note: The intent of this requirement is to specify that encryption of any protected data will not depend on a user electing to protect that data. The drive encryption specified in FDP_DSK_EXT.1 occurs transparently to the user and the decision to protect the data is outside the discretion of the user, which is a characteristic that distinguishes it from file encryption. The definition of protected data can be found in the glossary.

The cryptographic functions that perform the encryption/decryption of the data may be

provided by the Operational Environment (OE). Note that if this is the case, it is assumed that the OE implementation of AES is consistent with the behavior described in FCS_COP.1/SKC.

5.1.3 Security Management (FMT)

FMT_SMF.1 Specification of Management Functions

FMT_SMF.1.1

The TSF shall be capable of performing the following management functions: [

a. *change the DEK, as specified in FCS_CKM.1/DEK, when re-provisioning or when commanded*
b. *erase the DEK, as specified in FCS_CKM_EXT.6*
c. *initiate TOE firmware/software updates*
d. *[**selection**: no other functions, configure a password for an update, import a wrapped DEK, configure cryptographic functionality, disable key recovery functionality, securely update the public key needed for trusted update, configure the number of failed validation attempts required to trigger corrective behavior, configure the corrective behavior to issue 1 in the event of an excessive number of failed validation attempts, [**assignment**: other management functions provided by the TSF]]]*

Application Note: The intent of this requirement is to express the management capabilities that the TOE possesses. This means that the TOE must be able to perform the listed functions. “Configure cryptographic functionality” could include key management functions; for example, the BEV will be wrapped or encrypted, and the EE will need to unwrap or decrypt the BEV. In item (d), if no other management functions are provided (or claimed), then “no other functions” should be selected.

For the purposes of this document, key sanitization means to destroy the DEK, using one of the approved destruction methods. This applies to instances of the protected key that exist in non-volatile storage.

5.1.4 Protection of the TSF (FPT)

FPT_KYP_EXT.1 Protection of Key and Key Material

FPT_KYP_EXT.1.1

The TSF shall [**selection**:

- *not store keys in non-volatile memory*
- *only store keys in non-volatile memory when [**selection**: wrapped, as specified in FCS_COP.1/KeyWrap, encrypted, as specified in FCS_COP.1/KeyEnc, encrypted as specified in FCS_COP.1/KeyEncap]*
- *only store plaintext keys that meet any one of the following criteria [**selection**:*
  - *the plaintext key is not part of the key chain as specified in FCS_KYC_EXT.2*
  - *the plaintext key will no longer provide access to the encrypted data after initial provisioning*
  - *the plaintext key is a key split that is combined as specified in FCS_SMC_EXT.1, and the other half of the key split is [**selection**:*

- *wrapped as specified in FCS_COP.1/KeyWrap*
- *encrypted as specified in [**selection**: FCS_COP.1/KeyEnc, FCS_COP.1/KeyEncap]*
- *derived and not stored in non-volatile memory*

]

- *the non-volatile memory the key is stored on is located in an external storage device for use as an authorization factor*
- *the plaintext key is only used to provide additional cryptographic protection to other keys, such that disclosure of the plaintext key would not compromise the security of the keys being protected*

]

].

Application Note: The plaintext key storage in non-volatile memory is allowed for several reasons. If the keys exist within protected memory that is not user accessible on the TOE or OE, the only methods that allow it to play a security relevant role for protecting the BEV or the DEK are if it is a key split or providing additional layers of wrapping or encryption on keys that have already been protected.

If the TSF implements key wrapping, key encryption, or key encapsulation to maintain protected cryptographic key storage, then FCS_COP.1/KeyWrap, FCS_COP.1/KeyEnc, or FCS_COP.1/KeyEncap must be claimed. If the TSF implements submask combining to maintain protected cryptographic key storage, then FCS_SMC_EXT.1 must be claimed. These selection must align with FCS_KYC_EXT.2.

FPT_PWR_EXT.1 Power Saving States

FPT_PWR_EXT.1.1

The TSF shall define the following compliant power saving states: [**selection**: *S3*, *S4*, *G2(S5)*, *G3*, *D0*, *D1*, *D2*, *D3*, *[**assignment**: other power saving states]*].

Application Note: Power saving states S3, S4, G2(S5), G3, D0, D1, D2, and D3 are defined by the Advanced Configuration and Power Interface (ACPI) standard.

FPT_PWR_EXT.2 Timing of Power Saving States

FPT_PWR_EXT.2.1

For each compliant power saving state defined in FPT_PWR_EXT.1.1, the TSF shall enter the compliant power saving state when the following conditions occur: user-initiated request, [**selection**: *shutdown*, *user inactivity*, *request initiated by remote management system*, *[**assignment**: other conditions]*, *no other conditions*].

Application Note: If volatile memory is not erased as part of an unexpected power shutdown sequence then guidance documentation must define mitigation activities (e.g. how long users should wait after an unexpected power-down before volatile memory can be considered erased).

FPT_TST.1 TSF Self-Testing

FPT_TST.1.1

The TSF shall run a suite of the following self-tests [**selection**: *during initial start-up*, *at the conditions [before the function is first invoked]*] to demonstrate the correct operation of [**selection**: *[**assignment**: parts of TSF]*, *the TSF*]: [**assignment**: *list of self-tests run by the TSF*].

FPT_TST.1.2

The TSF shall provide authorized users with the capability to verify the integrity of [**selection**: *[**assignment**: parts of TSF data], TSF data*].

FPT_TST.1.3

The TSF shall provide authorized users with the capability to verify the integrity of [**selection**: *[**assignment**: parts of TSF], TSF*].

Application Note: This SFR is a required dependency of FCS_RBG.1 and the cryptographic requirements that are selectable in FCS_KYC_EXT.2. It is intended to require that any DRBG implemented by the TOE undergo health testing to ensure that the random bit generation functionality has not been degraded. If the TSF supports multiple DRBGs, this SFR should be iterated to describe the self-test behavior for each. The tests regarding cryptographic functions implemented in the TOE can be deferred, as long as the tests are performed before the function is invoked.

If any FCS_COP functions are implemented by the TOE, the TSS should describe the known answer self-tests for those functions.

FPT_TUD_EXT.1 Trusted Update

FPT_TUD_EXT.1.1

The TSF shall provide [*authorized users*] the ability to query the current version of the TOE [**selection**: *software, firmware*].

FPT_TUD_EXT.1.2

The TSF shall provide [*authorized users*] the ability to initiate updates to TOE [**selection**: *software, firmware*].

FPT_TUD_EXT.1.3

The TSF shall verify updates to the TOE [**selection**: *software, firmware*] using a [**selection**: *digital signature **as specified in FCS_COP.1/SigVer**, authenticated firmware update mechanism as described in FPT_FUA_EXT.1*] by the manufacturer prior to installing those updates.

Application Note: While this component requires the TOE to implement the update functionality itself, it is acceptable to perform the cryptographic checks using functionality available in the Operational Environment.

5.1.5 TOE Security Functional Requirements Rationale

The following rationale provides justification for each SFR for the TOE, showing that the SFRs are suitable to address the specified threats:

Table 5: SFR Rationale

Threat	Addressed by	Rationale
T.AUTHORIZATION_GUESSING	FCS_SNI_EXT.1	Mitigates this threat by requiring proper salts, which will prevent pre-computed attacks.
	FCS_VAL_EXT.1	Mitigates this threat by requiring several options for enforcing validation, such as key sanitization of the DEK or when a configurable number of failed validation attempts is reached within a 24 hour period. This mitigates brute force attacks.
T.CHOSEN_PLAINTEXT	FCS_COP.1/SKC	Mitigates this threat by ensuring the proper choice of encryption algorithm and mode.

	FCS_SNI_EXT.1	Mitigates this threat by ensuring proper handling of salts, nonces, and initialization vectors.
T.KEYING_MATERIAL_COMPROMISE	FCS_CKM.1/DEK	Mitigates this threat by ensuring that a sufficiently strong DEK is used to prevent its value from being determined.
	FCS_COP.1/SKC	Mitigates this threat by performing data encryption and decryption.
	FCS_CKM_EXT.6	Mitigates this threat by ensuring proper key material destruction.
	FCS_CKM.6/Power	Mitigates this threat by ensuring proper key material destruction for system power states.
	FCS_COP.1/Hash	Mitigates this threat by performing cryptographic hashing services.
	FCS_KYC_EXT.2	Mitigates this threat by requiring chaining of keys to protect the BEV.
	FCS_SNI_EXT.1	Mitigates this threat by requiring additional obfuscation to the protected key material by introducing IV's and salting.
	FCS_VAL_EXT.1	Mitigates this threat by defining methods for validation of the BEV and number of validation attempts.
	FMT_SMF.1	Mitigates this threat by ensuring the TSF provides the functions necessary to manage important aspects of the TOE including generating new keys.
	FPT_KYP_EXT.1	Mitigates this threat by requiring unwrapped key material is not stored in non-volatile memory.
	FPT_PWR_EXT.1	Mitigates this threat by requiring the TOE to meet a compliant power saving state that protects and/or destroys key material.
	FPT_PWR_EXT.2	Mitigates this threat by requiring the TOE to enter into a safe power saving state based on each condition.
	FCS_CKM.1/AKG (selection-based)	Mitigates this threat by requiring asymmetric key generation.
	FCS_CKM.6/KEK (optional)	Mitigates this threat by ensuring proper key cryptographic erase.
	FCS_CKM.1/SKG (implementation-dependent)	Mitigates this threat by requiring symmetric cryptographic key generation.
	FCS_CKM.6/GENHW (selection-based)	Mitigates this threat by ensuring proper key material destruction in general hardware.

	FCS_CKM.6/SW (selection-based)	Mitigates this threat by ensuring proper key material destruction within the software TOE and 3rd party storage.
	FCS_CKM.6/TOEHW (selection-based)	Mitigates this threat by ensuring proper key material destruction.
	FCS_COP.1/KeyedHash (selection-based)	Mitigates this threat by performing cryptographic keyed-hash message authentication.
	FCS_COP.1/KeyEnc (selection-based)	Mitigates this threat by performing key encryption and decryption.
	FCS_COP.1/KeyWrap (selection-based)	Mitigates this threat by performing key wrapping.
	FCS_CKM.5 (selection-based)	Mitigates this threat by ensuring strong key derivation methods.
	FCS_RBG.1 (selection-based)	Mitigates this threat by randomizing the generated keys in order to reduce the likelihood of guessing the future keys.
	FCS_RBG.2 (selection-based)	Mitigates this threat by ensuring that the TOE's DRBG is seeded with sufficient entropy to ensure the generation of strong cryptographic keys.
	FCS_RBG.3 (selection-based)	Mitigates this threat by ensuring that the TOE's DRBG is seeded with sufficient entropy to ensure the generation of strong cryptographic keys.
	FCS_RBG.4 (selection-based)	Mitigates this threat by ensuring that the TOE's DRBG is seeded with sufficient entropy to ensure the generation of strong cryptographic keys.
	FCS_RBG.5 (selection-based)	Mitigates this threat by ensuring that the TOE's DRBG is seeded with sufficient entropy to ensure the generation of strong cryptographic keys.
	FCS_SMC_EXT.1 (selection-based)	Mitigates this threat by obscuring the submasks via a XOR or hashing operation.
	FPT_FLS.1 (selection-based)	Mitigates this threat by ensuring that a malfunctioning DRBG function cannot be used to generate potentially insecure keys.
	FPT_TST.1 (selection-based)	Mitigates this threat by verifying the cryptographic functionality through the self testing functionality.
T.KEYSPACE_EXHAUST	FCS_CKM.1/DEK	Mitigates this threat by defining a sufficiently large keyspace such that keyspace exhaust to determine the value of the DEK is computationally infeasible.
	FCS_KYC_EXT.2	Mitigates this threat by ensuring all keys protecting the BEV are of the same strength.

	FCS_CKM.1/AKG (selection-based)	Mitigates this threat by requiring asymmetric key generation.
	FCS_CKM.1/SKG (implementation-dependent)	Mitigates this threat by requiring symmetric cryptographic key generation.
	FCS_RBG.1 (selection-based)	Mitigates this threat by ensuring that keys used for data encryption are generated using a secure DRBG.
	FCS_RBG.2 (selection-based)	Mitigates this threat by ensuring that the TOE's DRBG is seeded with sufficient entropy to ensure the generation of strong cryptographic keys.
	FCS_RBG.3 (selection-based)	Mitigates this threat by ensuring that the TOE's DRBG is seeded with sufficient entropy to ensure the generation of strong cryptographic keys.
	FCS_RBG.4 (selection-based)	Mitigates this threat by ensuring that the TOE's DRBG is seeded with sufficient entropy to ensure the generation of strong cryptographic keys.
	FCS_RBG.5 (selection-based)	Mitigates this threat by ensuring that the TOE's DRBG is seeded with sufficient entropy to ensure the generation of strong cryptographic keys.
	FPT_FLS.1 (selection-based)	Mitigates this threat by ensuring that a malfunctioning DRBG function cannot be used to generate potentially insecure keys.
	FPT_TST.1 (selection-based)	Mitigates this threat by verifying the cryptographic functionality through the self testing functionality.
T.KNOWN_PLAINTEXT	FCS_COP.1/SKC	Mitigates this threat by ensuring the proper choice of encryption algorithm and mode.
	FCS_SNI_EXT.1	Mitigates this threat by ensuring proper handling of salts, nonces, and initialization vectors.
T.UNAUTHORIZED_DATA_ACCESS	FCS_COP.1/SKC	Mitigates this threat by defining proper AES encryption.
	FCS_SNI_EXT.1	Mitigates this threat by ensuring proper nonces and IVs are used in the encryption of data.
	FCS_VAL_EXT.1	Mitigates this threat by verifying the correct authentication and limits attempts to decrypt the data.
	FDP_DSK_EXT.1	Mitigates this threat by ensuring the TOE performs full drive encryption, which includes all protected data.
	FMT_SMF.1	Mitigates this threat by ensuring the TSF provides the functions necessary to manage important aspects of the TOE including requests to change and erase the DEK.

	FPT_PWR_EXT.1	Mitigates this threat by defining what power states are compliant for the TOE.
	FPT_PWR_EXT.2	Mitigates this threat by defining conditions in which the TOE will enter a compliant power state. These requirements ensure the device is secure if lost in a compliant power state.
T.UNAUTHORIZED_ FIRMWARE_MODIFY	FPT_TUD_EXT.1	Mitigates this threat by providing a mechanism used to initiate updates where the authenticity and integrity of the update can be verified as part of the update process.
	FPT_FUA_EXT.1 (selection-based)	Mitigates this threat by ensuring existing firmware cannot be modified unless replaced with a valid update initiated as part of FPT_TUD_EXT.1
	FCS_COP.1/Hash	Mitigates this threat by defining the cryptographic functions that can be used to validate the authenticity and integrity of firmware updates as defined by FPT_FUA_EXT.1.
	FCS_COP.1/SigVer	Mitigates this threat by defining the cryptographic functions that can be used to validate the authenticity and integrity of firmware updates as defined by FPT_FUA_EXT.1.
T.UNAUTHORIZED_ UPDATE	FCS_COP.1/SigVer	Mitigates this threat by defining the signature function that is used to verify updates.
	FMT_SMF.1	Mitigates this threat by ensuring the TSF provides the functions necessary to manage important behavior of the TOE which includes the initiation of system firmware/software updates.
	FPT_TUD_EXT.1	Mitigates this threat by providing authorized users the ability to query the current version of the TOE software/firmware, initiate updates, and verify updates prior to installation using a manufacturer digital signature.
	FPT_FAC_EXT.1 (optional)	Mitigates this threat by providing additional security by only allowing an update to be initiated by an authorized user.
	FPT_RBP_EXT.1 (optional)	Mitigates this threat by protecting against a malicious or inadvertent downgrade of the software/firmware to an earlier version that may have security flaws not present in the more recent version.

5.2 Security Assurance Requirements

This cPP identifies the Security Assurance Requirements (SARs) to frame the extent to which the evaluator assesses the documentation applicable for the evaluation and performs independent testing. Individual evaluation activities to be performed are specified within each SFR.

Note to ST authors: There is a selection in the ASE_TSS that must be completed. One cannot simply reference the SARs in this cPP.

The general model for evaluation of TOEs against STs written to conform to this cPP is as follows: after the ST has been approved for evaluation, the ITSEF will obtain the TOE, supporting environmental IT (if required), and the administrative/user guides for the TOE. The ITSEF is expected to perform actions mandated by the Common Evaluation Methodology (CEM) for the ASE and ALC SARs. The ITSEF also performs the Evaluation Activities contained within the SD, which are intended to be an interpretation of the other CEM assurance requirements as they apply to the specific technology instantiated in the TOE. The Evaluation Activities that are captured in the SD also provide clarification as to what the developer needs to provide to demonstrate the TOE is compliant with the cPP.

Table 6: TOE Security Assurance Requirements

Functional Class	Functional Components
Security Target (ASE)	Conformance Claims (ASE_CCL.1)
	Extended Components Definition (ASE_ECD.1)
	ST Introduction (ASE_INT.1)
	Security Objectives for the Operational Environment (ASE_OBJ.1)
	Stated Security Requirements (ASE_REQ.1)
	Security Problem Definition (ASE_SPD.1)
	TOE Summary Specification (ASE_TSS.1)
Development (ADV)	Basic Functional Specification (ADV_FSP.1)
Guidance Documents (AGD)	Operational User Guidance (AGD_OPE.1)
	Preparative Procedures (AGD_PRE.1)
Life Cycle Support (ALC)	Labeling of the TOE (ALC_CMC.1)
	TOE CM Coverage (ALC_CMS.1)
Tests (ATE)	Independent Testing – Sample (ATE_IND.1)
Vulnerability Assessment (AVA)	Vulnerability Survey (AVA_VAN.1)

5.2.1 ASE: Security Target

The ST is evaluated as per ASE activities defined in the CEM. In addition, there may be Evaluation Activities specified within the SD that call for necessary descriptions to be included in the TSS that are specific to the TOE technology type.

The SFRs in this cPP allow for conformant implementations to incorporate a wide range of acceptable key management approaches as long as basic principles are satisfied. Given the criticality of the key management scheme, this cPP requires the developer to provide a detailed description of their key management implementation. This information can be submitted as an appendix to the ST and marked proprietary, as this level of detailed information is not expected to be made publicly available. See Appendix E for details on the expectation of the developer's Key Management Description

In addition, if the TOE includes a random bit generator Appendix D provides a description of the information expected to be provided regarding the quality of the entropy.

ASE_TSS.1.1C The TOE summary specification shall describe how the TOE meets each SFR, **including a Key Management Description (Appendix E), and *[selection: Entropy Essay, list of all of 3rd party software libraries (including version numbers), 3rd party hardware components (including model/version numbers), no other cPP specified proprietary documentation]*.**

5.2.2 ADV: Development

The design information about the TOE is contained in the guidance documentation available to the end user as well as the TSS portion of the ST, and any additional information required by this cPP that is not to be made public (e.g., Entropy Essay).

ADV_FSP.1 Basic Functional Specification (ADV_FSP.1)

The functional specification describes the TOE Security Functions Interfaces (TSFIs). It is not necessary to have a formal or complete specification of these interfaces. Additionally, because TOEs conforming to this cPP may have interfaces to the Operational Environment that are not directly invoked by TOE users, there is little point specifying that such interfaces be described in and of themselves since only indirect testing of such interfaces may be possible. For this cPP, the evaluation activities for this family focus on understanding the interfaces presented in the TSS in response to the functional requirements and the interfaces presented in the AGD documentation. No additional "functional specification" documentation is necessary to satisfy the evaluation activities specified.

The evaluation activities are associated with the applicable SFRs. Since these are directly associated with the SFRs, the tracing in element ADV_FSP.1.2D is implicitly already done and no additional documentation is necessary.

Developer action elements:

ADV_FSP.1.1D

The developer shall provide a functional specification.

ADV_FSP.1.2D

The developer shall provide a tracing from the functional specification to the SFRs.

Application Note: As indicated in the introduction to this section, the functional specification is comprised of the information contained in the AGD_OPE and AGD_PRE documentation. The developer may reference a website accessible to application developers and the evaluator. The evaluation activities in the functional requirements point to evidence that should exist in the documentation and TSS section; since these are directly associated with the SFRs, the tracing in element ADV_FSP.1.2D is implicitly already done and no additional documentation is necessary.

Content and presentation elements:

ADV_FSP.1.1C

The functional specification shall describe the purpose and method of use for each SFR-enforcing and SFR-supporting TSFI.

ADV_FSP.1.2C

The functional specification shall identify all parameters associated with each SFR-enforcing and SFR-supporting TSFI.

ADV_FSP.1.3C

The functional specification shall provide rationale for the implicit categorization of interfaces as SFR-non-interfering.

ADV_FSP.1.4C

The tracing shall demonstrate that the SFRs trace to TSFIs in the functional specification.

Evaluator action elements:

ADV_FSP.1.1E

The evaluator shall confirm that the information provided meets all requirements for content and presentation of evidence.

ADV_FSP.1.2E

The evaluator shall determine that the functional specification is an accurate and complete instantiation of the SFRs.

5.2.3 AGD: Guidance Documentation

The guidance documents will be provided with the ST. Guidance must include a description of how the IT personnel verifies that the Operational Environment can fulfill its role for the security functionality. The documentation should be in an informal style and readable by the IT personnel.

Guidance must be provided for every operational environment that the product supports as claimed in the ST. This guidance includes:

- Instructions to successfully install the TSF in that environment; and
- Instructions to manage the security of the TSF as a product and as a component of the larger operational environment
- Instructions to provide a protected administrative capability.

Guidance pertaining to particular security functionality must also be provided; requirements on such guidance are contained in the evaluation activities

AGD_OPE.1 Operational User Guidance (AGD_OPE.1)

The operational user guidance does not have to be contained in a single document. Guidance to users, administrators, and integrators can be spread among documents or web pages.

The developer should review the evaluation activities to ascertain the specifics of the guidance that the evaluator will be checking for. This will provide the necessary information for the preparation of acceptable guidance.

Developer action elements:

AGD_OPE.1.1D

The developer shall provide operational user guidance.

Application Note: The operational user guidance does not have to be contained in a single document. Guidance to users, administrators and application developers can be spread among documents or web pages. Where appropriate, the guidance documentation is expressed in the eXtensible Configuration Checklist Description Format (XCCDF) to support security automation. Rather than repeat information here, the developer should review the evaluation activities for this component to ascertain the specifics of the guidance that the evaluator will be checking for. This will provide the necessary information for the preparation of acceptable guidance.

Content and presentation elements:

AGD_OPE.1.1C

The operational user guidance shall describe, for each user role, the user-accessible functions and privileges that should be controlled in a secure processing environment, including appropriate warnings.

Application Note: User and administrator are to be considered in the definition of user role.

AGD_OPE.1.2C

The operational user guidance shall describe, for each user role, how to use the available interfaces provided by the TOE in a secure manner.

AGD_OPE.1.3C

The operational user guidance shall describe, for each user role, the available functions and interfaces, in particular all security parameters under the control of the user, indicating secure values as appropriate.

AGD_OPE.1.4C

The operational user guidance shall, for each user role, clearly present each type of security-relevant event relative to the user-accessible functions that need to be performed, including changing the security characteristics of entities under the control of the TSF.

AGD_OPE.1.5C

The operational user guidance shall identify all possible modes of operation of the TOE (including operation following failure or operational error), their consequences, and implications for maintaining secure operation.

AGD_OPE.1.6C

The operational user guidance shall, for each user role, describe the security measures to be followed in order to fulfill the security objectives for the operational environment as described in the ST.

AGD_OPE.1.7C

The operational user guidance shall be clear and reasonable.

Evaluator action elements:

AGD_OPE.1.1E

The evaluator shall confirm that the information provided meets all requirements for content and presentation of evidence.

AGD_PRE.1 Preparative Procedures (AGD_PRE.1)

As with the operational guidance, the developer should look to the Evaluation Activities to determine the required content with respect to preparative procedures.

Developer action elements:

AGD_PRE.1.1D

The developer shall provide the TOE, including its preparative procedures.

Application Note: As with the operational guidance, the developer should look to the evaluation activities to determine the required content with respect to preparative procedures.

Content and presentation elements:

AGD_PRE.1.1C

The preparative procedures shall describe all the steps necessary for secure acceptance of the delivered TOE in accordance with the developer's delivery procedures.

AGD_PRE.1.2C

The preparative procedures shall describe all the steps necessary for secure installation of the TOE and for the secure preparation of the operational environment in accordance with the security objectives for the operational environment as described in the ST.

Evaluator action elements:

AGD_PRE.1.1E

The evaluator shall confirm that the information provided meets all requirements for content and presentation of evidence.

AGD_PRE.1.2E

The evaluator shall apply the preparative procedures to confirm that the TOE can be prepared securely for operation.

5.2.4 Class ALC: Life-cycle Support

At the assurance level provided for TOEs conformant to this cPP, life-cycle support is limited to end-user-visible aspects of the life-cycle, rather than an examination of the TOE vendor's development and configuration management process. This is not meant to diminish the critical role that a developer's practices play in contributing to the overall trustworthiness of a product; rather, it is a reflection on the information to be made available for evaluation at this assurance level.

ALC_CMC.1 Labelling of the TOE (ALC_CMC.1)

This component is targeted at identifying the TOE such that it can be distinguished from other products or versions from the same vendor and can be easily specified when being procured by an end user. The evaluator performs the CEM work units associated with ALC_CMC.1.

Developer action elements:

ALC_CMC.1.1D

The developer shall provide the TOE and a reference for the TOE.

Content and presentation elements:

ALC_CMC.1.1C

The application shall be labeled with a unique reference.

Application Note: Unique reference information includes:

- Application Name
- Application Version
- Application Description
- Platform on which Application Runs
- Software Identification (SWID) tags, if available

Evaluator action elements:

ALC_CMC.1.1E

The evaluator shall confirm that the information provided meets all requirements for content and presentation of evidence.

ALC_CMS.1 TOE CMS Coverage (ALC_CMS.1)

Given the scope of the TOE and its associated evaluation evidence requirements, the evaluator performs the CEM work units associated with ALC_CMS.1.

Developer action elements:

ALC_CMS.1.1D

The developer shall provide a configuration list for the TOE.

Content and presentation elements:

ALC_CMS.1.1C

The configuration list shall include the following: the TOE itself; and the evaluation evidence required by the SARs.

ALC_CMS.1.2C

The configuration list shall uniquely identify the configuration items.

Evaluator action elements:

ALC_CMS.1.1E

The evaluator shall confirm that the information provided meets all requirements for content and presentation of evidence.

5.2.5 Class ATE: Tests

Testing is specified for functional aspects of the system as well as aspects that take advantage of design or implementation weaknesses. The former is done through the ATE_IND family, while the latter is through the AVA_VAN family. For this cPP, testing is based on advertised functionality and interfaces with dependency on the availability of design information. One of the primary outputs of the evaluation process is the test report as specified in the following requirements.

ATE_IND.1 Independent Testing – Conformance (ATE_IND.1)

Testing is performed to confirm the functionality described in the TSS as well as the operational guidance (includes “evaluated configuration” instructions). The focus of the testing is to confirm that the requirements specified in Section 5 are being met. The Evaluation Activities in the SD identify the specific testing activities necessary to verify compliance with the SFRs. The evaluator produces a test report documenting the plan for and results of testing, as well as coverage arguments focused on the platform/TOE combinations that are claiming conformance to this cPP.

Developer action elements:

ATE_IND.1.1D

The developer shall provide the TOE for testing.

Application Note: The developer must provide at least one product instance of the TOE for complete testing on at least one platform regardless of equivalency. See the Equivalency Appendix for more details.

Content and presentation elements:

ATE_IND.1.1C

The TOE shall be suitable for testing.

Evaluator action elements:

ATE_IND.1.1E

The evaluator *shall confirm* that the information provided meets all requirements for content and presentation of evidence.

ATE_IND.1.2E

The evaluator shall test a subset of the TSF to confirm that the TSF operates as specified.

Application Note: The evaluator should test the application on the most current fully patched version of the platform.

5.2.6 Class AVA: Vulnerability Assessment

For the current generation of this cPP, the iTC is expected to survey open sources to discover what vulnerabilities have been discovered in these types of products and provide that content into the AVA_VAN discussion. In most cases, these vulnerabilities will require sophistication beyond that of a basic attacker. This information will be used in the development of future Protection Profiles.

If the TOE is a Network Attached Storage (NAS) device, the evaluator shall verify as part of the vulnerability assessment that remote management services are either not present or can be fully disabled.

AVA_VAN.1 Vulnerability Survey (AVA_VAN.1)

Vulnerability Analysis

Sources of Vulnerability Information

CEM Work Unit AVA_VAN.1-3 is supplemented here to provide a better-defined set of flaws to investigate and procedures to follow based on this particular technology. Terminology used is based on the flaw hypothesis methodology, where the evaluation team hypothesizes flaws and then either proves or disproves those flaws (a flaw is equivalent to a “potential vulnerability” as used in the CEM). Flaws are categorized into four “types” depending on how they are formulated:

1. A list of flaw hypotheses applicable to the technology described by the cPP derived from public sources as documented in the Type 1 Hypotheses section—this fixed set has been agreed to by the iTC. Additionally, this will be supplemented with entries for a set of public sources (as indicated below) that are directly applicable to the TOE or its identified components (as defined by the process in the Type 1 Hypotheses section below); this is to ensure that the evaluators include in their assessment applicable entries that have been discovered since the cPP was published;
2. A list of flaw hypotheses contained in this document that are derived from lessons learned specific to that technology and other iTC input (that might be derived from other open sources and vulnerability databases, for example) as documented in the Type 2 Hypotheses section;
3. A list of flaw hypotheses derived from information available to the evaluators; this includes the baseline evidence provided by the vendor described in this document (documentation associated with EAs, documentation described in the Vulnerability Survey section), as well as other information (public and/or based on evaluator experience) as documented in the Type 3 Hypotheses section; and
4. A list of flaw hypotheses that are generated through the use of iTC-defined tools (e.g., nmap, protocol testers) and their application is specified in the Type 4 Hypotheses section.

Type 1 Hypotheses-Public-Vulnerability-Based

The following list of public sources of vulnerability information was selected by the iTC:

a. Search Common Vulnerabilities and Exposures: http://cve.mitre.org/cve/
b. Search the National Vulnerability Database: https://nvd.nist.gov/
c. Search US-CERT: http://www.kb.cert.org/vuls/html/search

The list of sources above was searched with the following search terms:

- General (for all)
    - Product name
    - Underlying components (e.g., OS, software libraries (crypto libraries), chipsets)
    - Drive encryption, disk encryption
    - Key destruction and sanitization

- For Software FDE (AA or EE)
    - Key caching

In order to successfully complete this activity, the evaluator will use the developer provided list of all of third party library information that is used as part of their product, along with the version and any other identifying information (this is required in the cPP as part of the ASE_TSS.1.1C requirement). This applies to hardware (including chipsets, etc.) that a vendor utilizes as part of their TOE. This TOE-unique information will be used in the search terms the evaluator uses in addition to those listed above.

The evaluator will also consider the requirements that are chosen and the appropriate guidance that is tied to each requirement.

In order to supplement this list, the evaluators shall also perform a search on the sources listed above to determine a list of potential flaw hypotheses that are more recent that the publication date of the cPP, and those that are specific to the TOE and its components as specified by the additional documentation mentioned above. Any duplicates – either in a specific entry, or in the flaw hypothesis that is generated from an entry from the same or a different source – can be noted and removed from consideration by the evaluation team.

As part of type 1 flaw hypothesis generation for the specific components of the TOE, the evaluator shall also search the component manufacturer's websites to determine if flaw hypotheses can be generated on this basis (for instance, if security patches have been released for the version of the component being evaluated, the subject of those patches may form the basis for a flaw hypothesis).

Type 2 Hypotheses-iTC-Sourced

There are no type 2 hypotheses.

Type 3 Hypotheses-Evaluation-Team-Generated

The iTC has leveraged the expertise of the developers and the evaluation labs to diligently develop the appropriate search terms and vulnerability databases. They have also thoughtfully considered the iTC-sourced hypotheses the evaluators should use based upon the applicable use case and the threats to be mitigated by the SFRs. Therefore, it is the intent of the iTC, for the evaluation to focus all effort on the Type 1 and Type 2 Hypotheses and has decided that Type 3 Hypotheses are not necessary.

However, if the evaluators discover a Type 3 potential flaw that they believe should be considered, they should work with their Certification Body to determine the feasibility of pursuing the hypothesis. The Certification Body may determine whether the potential flaw hypotheses is worth submitting to the iTC for consideration as Type 2 hypotheses in future drafts of the cPP/SD.

Type 4 Hypotheses-Evaluation-Team-Generated

The iTC has called out several tools that should be used during the Type 2 hypotheses process. Therefore, the use of any tools is covered within the Type 2 construct and the iTC does not see any additional tools that are necessary. The use case for Version 2 of this cPP is rather straightforward – the device is found in a powered down state and has not been subjected to revisit/evil maid attacks. Since that is the use case, the iTC has also assumed there is a trusted channel between the AA and EE. Since the use case is so narrow, and is not a typical model for penetration or fuzzing testing, the normal types of testing do not apply. Therefore, the relevant types of tools are referenced in Type 2.

Process for Evaluator Vulnerability Analysis

As flaw hypotheses are generated from the activities described above, the evaluation team will disposition them; that is, attempt to prove, disprove, or determine the non-applicability of the hypotheses. This process is as follows. The evaluator will refine each flaw hypothesis for the TOE and attempt to disprove it using the

information provided by the developer or through penetration testing. During this process, the evaluator is free to interact directly with the developer to determine if the flaw exists, including requests to the developer for additional evidence (e.g., detailed design information, consultation with engineering staff); however, the CB should be included in these discussions. Should the developer object to the information being requested as being not compatible with the overall level of the evaluation activity/cPP and cannot provide evidence otherwise that the flaw is disproved, the evaluator prepares an appropriate set of materials as follows:

- The source documents used in formulating the hypothesis, and why it represents a potential compromise against a specific TOE function;
- An argument why the flaw hypothesis could not be proven or disproved by the evidence provided so far; and
- The type of information required to investigate the flaw hypothesis further.

The Certification Body (CB) will then either approve or disapprove the request for additional information. If approved, the developer provides the requested evidence to disprove the flaw hypothesis (or, of course, acknowledge the flaw).

For each hypothesis, the evaluator will note whether the flaw hypothesis has been successfully disproved, successfully proven to have identified a flaw, or requires further investigation. It is important to have the results documented as outlined in the Reporting section below. If the evaluator finds a flaw, the evaluator must report these flaws to the developer. All reported flaws must be addressed as follows:

If the developer confirms that the flaw exists and that it is exploitable at Basic Attack Potential, then a change is made by the developer, and the resulting resolution is agreed by the evaluator and noted as part of the evaluation report.

If the developer, the evaluator, and the CB agree that the flaw is exploitable only above Basic Attack Potential and does not require resolution for any other reason, then no change is made and the flaw is noted as a residual vulnerability in the CB-internal report (ETR).

If the developer and evaluator agree that the flaw is exploitable only above Basic Attack Potential, but it is deemed critical to fix because of technology-specific or cPP-specific aspects such as typical use cases or operational environments, then a change is made by the developer, and the resulting resolution is agreed by the evaluator and noted as part of the evaluation report.

Disagreements between evaluator and vendor regarding questions of the existence of a flaw, its attack potential, or whether it should be deemed critical to fix are resolved by the CB.

Any testing performed by the evaluator shall be documented in the test report as outlined in the Reporting section below.

As indicated in the Reporting section, the public statement with respect to vulnerability analysis that is performed on TOEs conformant to the cPP is constrained to coverage of flaws associated with Types 1 and 2 (defined in the Sources of Vulnerability Information section) flaw hypotheses only. The fact that the iTC generates these candidate hypotheses indicates these must be addressed.

Reporting

The evaluators shall produce two reports on the testing effort; one that is public-facing (that is, included in the non-proprietary evaluation report, which is a subset of the Evaluation Technical Report (ETR)), and the complete ETR that is delivered to the overseeing CB.

The public-facing report contains:

- The flaw identifiers returned when the procedures for searching public sources were followed according to instructions in the Type 1 Hypotheses section;
- A statement that the evaluators have examined the Type 1 flaw hypotheses specified in this document in the Type 1 Hypotheses section (i.e. the flaws listed in the previous bullet) and the Type 2 flaw

hypotheses specified in the the Type 2 Hypotheses section

No other information is provided in the public-facing report.

The internal CB report contains, in addition to the information in the public-facing report:

- a list of all of the flaw hypotheses generated (cf. AVA_VAN.1-4);
- the evaluator penetration testing effort, outlining the testing approach, configuration, depth and results (cf. AVA_VAN.1-9);
- all documentation used to generate the flaw hypotheses (in identifying the documentation used in coming up with the flaw hypotheses, the evaluation team must characterize the documentation so that a reader can determine whether it is strictly required in this document, and the nature of the documentation (design information, developer engineering notebooks, etc.));
- the evaluator shall report all exploitable vulnerabilities and residual vulnerabilities, detailing for each:
    a. its source (e.g. CEM activity being undertaken when it was conceived, known to the evaluator, read in a publication)
    b. the SFRs not met;
    c. a description;
    d. whether it is exploitable in its operational environment or not (i.e. exploitable or residual).
    e. the amount of time, level of expertise, level of knowledge of the TOE, level of opportunity and the equipment required to perform the identified vulnerabilities (cf. AVA_VAN.1-11);
    f. how each flaw hypothesis was resolved (this includes whether the original flaw hypothesis was confirmed or disproved, and any analysis relating to whether a residual vulnerability is exploitable by an attacker with Basic Attack Potential) (cf. AVA_VAN.1-10); and
    g. in the case that actual testing was performed in the investigation (either as part of flaw hypothesis generation using tools specified by the iTC in the Type 4 Hypotheses section, or in proving or disproving a particular flaw) the steps followed in setting up the TOE (and any required test equipment); executing the test; post-test procedures; and the actual results (to a level of detail that allow repetition of the test, including the following:
        - identification of the potential vulnerability the TOE is being tested for;
        - instructions to connect and setup all required test equipment as required to conduct the penetration test;
        - instructions to establish all penetration test prerequisite initial conditions;
        - instructions to stimulate the TSF;
        - instructions for observing the behaviour of the TSF;
        - descriptions of all expected results and the necessary analysis to be performed on the observed behaviour for comparison against expected results;
        - instructions to conclude the test and establish the necessary post-test state for the TOE. (cf. AVA_VAN.1-6, AVA_VAN.1-8).

Developer action elements:

AVA_VAN.1.1D

The developer shall provide the TOE for testing.

Content and presentation elements:

AVA_VAN.1.1C

The application shall be suitable for testing.

Application Note: Suitability for testing means not being obfuscated or packaged in such a way as to disrupt either static or dynamic analysis by the evaluator.

Evaluator action elements:

AVA_VAN.1.1E

The evaluator shall confirm that the information provided meets all requirements for content and presentation of evidence.

AVA_VAN.1.2E

The evaluator shall perform a search of public domain sources to identify potential vulnerabilities in the TOE.

Application Note: Public domain sources include the Common Vulnerabilities and Exposures (CVE) dictionary for publicly known vulnerabilities. Public domain sources also include sites which provide free checking of files for viruses.

AVA_VAN.1.3E

The evaluator shall conduct penetration testing, based on the identified potential vulnerabilities, to determine that the TOE is resistant to attacks performed by an attacker possessing Basic attack potential.

Appendix A - Optional Requirements

As indicated in the introduction to this PP, the baseline requirements (those that must be performed by the TOE) are contained in the body of this PP. This appendix contains three other types of optional requirements:

The first type, defined in Appendix A.1 Strictly Optional Requirements, are strictly optional requirements. If the TOE meets any of these requirements the vendor is encouraged to claim the associated SFRs in the ST, but doing so is not required in order to conform to this PP.

The second type, defined in Appendix A.2 Objective Requirements, are objective requirements. These describe security functionality that is not yet widely available in commercial technology. Objective requirements are not currently mandated by this PP, but will be mandated in the future. Adoption by vendors is encouraged, but claiming these SFRs is not required in order to conform to this PP.

The third type, defined in Appendix A.3 Implementation-dependent Requirements, are Implementation-dependent requirements. If the TOE implements the product features associated with the listed SFRs, either the SFRs must be claimed or the product features must be disabled in the evaluated configuration.

A.1 Strictly Optional Requirements

A.1.1 Class ALC: Life-cycle Support

ALC_FLR.1 Basic Flaw Remediation (ALC_FLR.1)

This SAR is optional and may be claimed at the ST-Author's discretion.

Developer action elements:

ALC_FLR.1.1D

The developer shall document and provide flaw remediation procedures addressed to TOE developers.

Content and presentation elements:

ALC_FLR.1.1C

The flaw remediation procedures documentation shall describe the procedures used to track all reported security flaws in each release of the TOE.

ALC_FLR.1.2C

The flaw remediation procedures shall require that a description of the nature and effect of each security flaw be provided, as well as the status of finding a correction to that flaw.

ALC_FLR.1.3C

The flaw remediation procedures shall require that corrective actions be identified for each of the security flaws.

ALC_FLR.1.4C

The flaw remediation procedures documentation shall describe the methods used to provide flaw information, corrections and guidance on corrective actions to TOE users.

Evaluator action elements:

ALC_FLR.1.1E

The evaluator shall confirm that the information provided meets all requirements for content and presentation of evidence.

ALC_FLR.2 Flaw Reporting Procedures (ALC_FLR.2)

This SAR is optional and may be claimed at the ST-Author's discretion.

Developer action elements:

ALC_FLR.2.1D

The developer shall document and provide flaw remediation procedures addressed to TOE developers.

ALC_FLR.2.2D

The developer shall establish a procedure for accepting and acting upon all reports of security flaws and requests for corrections to those flaws.

ALC_FLR.2.3D

The developer shall provide flaw remediation guidance addressed to TOE users.

Content and presentation elements:

ALC_FLR.2.1C

The flaw remediation procedures documentation shall describe the procedures used to track all reported security flaws in each release of the TOE.

ALC_FLR.2.2C

The flaw remediation procedures shall require that a description of the nature and effect of each security flaw be provided, as well as the status of finding a correction to that flaw.

ALC_FLR.2.3C

The flaw remediation procedures shall require that corrective actions be identified for each of the security flaws.

ALC_FLR.2.4C

The flaw remediation procedures documentation shall describe the methods used to provide flaw information, corrections and guidance on corrective actions to TOE users.

ALC_FLR.2.5C

The flaw remediation procedures shall describe a means by which the developer receives from TOE users reports and enquiries of suspected security flaws in the TOE.

ALC_FLR.2.6C

The procedures for processing reported security flaws shall ensure that any reported flaws are remediated and the remediation procedures issued to TOE users.

ALC_FLR.2.7C

The procedures for processing reported security flaws shall provide safeguards that any corrections to these security flaws do not introduce any new flaws.

ALC_FLR.2.8C

The flaw remediation guidance shall describe a means by which TOE users report to the developer any suspected security flaws in the TOE.

Evaluator action elements:

ALC_FLR.2.1E

The evaluator shall confirm that the information provided meets all requirements for content and presentation of evidence.

ALC_FLR.3 Systematic Flaw Remediation (ALC_FLR.3)

This SAR is optional and may be claimed at the ST-Author's discretion.

Developer action elements:

ALC_FLR.3.1D

The developer shall document and provide flaw remediation procedures addressed to TOE developers.

ALC_FLR.3.2D

The developer shall establish a procedure for accepting and acting upon all reports of security flaws and requests for corrections to those flaws.

ALC_FLR.3.3D

The developer shall provide flaw remediation guidance addressed to TOE users.

Content and presentation elements:

ALC_FLR.3.1C

The flaw remediation procedures documentation shall describe the procedures used to track all reported security flaws in each release of the TOE.

ALC_FLR.3.2C

The flaw remediation procedures shall require that a description of the nature and effect of each security flaw be provided, as well as the status of finding a correction to that flaw.

ALC_FLR.3.3C

The flaw remediation procedures shall require that corrective actions be identified for each of the security flaws.

ALC_FLR.3.4C

The flaw remediation procedures documentation shall describe the methods used to provide flaw information, corrections and guidance on corrective actions to TOE users.

ALC_FLR.3.5C

The flaw remediation procedures shall describe a means by which the developer receives from TOE users reports and enquiries of suspected security flaws in the TOE.

ALC_FLR.3.6C

The flaw remediation procedures shall include a procedure requiring timely response and the automatic distribution of security flaw reports and the associated corrections to registered users who might be affected by the security flaw.

ALC_FLR.3.7C

The procedures for processing reported security flaws shall ensure that any reported flaws are remediated and the remediation procedures issued to TOE users.

ALC_FLR.3.8C

The procedures for processing reported security flaws shall provide safeguards that any corrections to these security flaws do not introduce any new flaws.

ALC_FLR.3.9C

The flaw remediation guidance shall describe a means by which TOE users report to the developer any suspected security flaws in the TOE.

ALC_FLR.3.10C

The flaw remediation guidance shall describe a means by which TOE users may register with the developer, to be eligible to receive security flaw reports and corrections.

ALC_FLR.3.11C

The flaw remediation guidance shall identify the specific points of contact for all reports and enquiries about security issues involving the TOE.

Evaluator action elements:

ALC_FLR.3.1E

The evaluator shall confirm that the information provided meets all requirements for content and presentation of evidence.

A.1.2 Protection of the TSF (FPT)

FPT_FAC_EXT.1 Firmware / Software Access Control

FPT_FAC_EXT.1.1

The TSF shall require [**selection**: *a password, a unique value printed on the device, a authorized user action*] before the [**selection**: *software, firmware*] update proceeds.

Application Note: Before an update takes place, the TSF owner will authorize the update by providing either a unique value (for example, a serial number) that is printed on the drive, a password (which should be administratively configurable as defined in FMT_SMF.1), or perform a specific action as an authorized user. If applicable, it is assumed that physical presence to the drive is limited to authorized personnel. If the correct value is not provided, the update will not take place. The values are intended to be unique per drive so they cannot be easily guessed.

FPT_RBP_EXT.1 Rollback Protection

FPT_RBP_EXT.1.1

The TSF shall verify that the new update is not downgrading to a lower security version number by [**assignment**: *method of verifying the security version number is the same as or higher than the currently installed version*].

FPT_RBP_EXT.1.2

The TSF shall generate and return an error code if the attempted update package is detected to be an invalid version.

Application Note: This requirement prevents an unauthorized rollback of the update to an earlier authentic version. This mitigates against unknowing installation of an earlier authentic version that may have a security weakness. It is expected that vendors will increase security version numbers with each new update package.

For FPT_RBP_EXT.1.1 the purpose is to verify that the new package has a security version number equal to or larger than the security version number of currently installed package.

The administrator guidance would include instructions for the administrator to configure the rollback protection mechanism, if appropriate.

The security version may be different from other parts that undergo update by the TSF.

A.2 Objective Requirements

This PP does not define any Objective requirements.

A.3 Implementation-dependent Requirements

A.3.1 Cryptographic Support (FCS)

FCS_CKM.1/SKG Cryptographic Key Generation (Symmetric Keys)

This component must be included in the ST if the TOE implements any of the following features:

- ***Symmetric Key Generation Support***

FCS_CKM.1.1/SKG

The TSF shall generate **symmetric** cryptographic keys **using a Random Bit Generator as specified in FCS_RBG.1** and specified cryptographic key sizes *256 bit* that meet the following: [*no standard*].

Application Note: Symmetric keys may be used to generate keys along the key chain. This applies to any instance in where the TSF DRBG is referenced for key generation where the TSF generates or re-generates key encryption or key wrapping keys as part of deriving a key (as in FCS_KYC_EXT.2)

Appendix B - Selection-based Requirements

As indicated in the introduction to this PP, the baseline requirements (those that must be performed by the TOE or its underlying platform) are contained in the body of this PP. There are additional requirements based on selections in the body of the PP: if certain selections are made, then additional requirements below must be included.

B.1 Cryptographic Support (FCS)

FCS_CKM.1/AKG Cryptographic Key Generation (Asymmetric Keys)

The inclusion of this selection-based component depends upon selection in:

- ***FCS_COP.1.1/KeyEncap***

FCS_CKM.1.1/AKG

The TSF shall generate **asymmetric** cryptographic keys in accordance with a specified cryptographic key generation algorithm [**selection**:

- *CNSA 2.0 Compliant Algorithms: [**selection**:*
  - ***Module-Lattice-Based Key-Encapsulation Mechanism** using the parameter set ML-KEM-1024*
  - ***Module-Lattice-Based Digital Signature Algorithm** using the parameter set ML-DSA-87*

  ]
- *CNSA 1.0 Compliant Algorithms: [**selection**:*
  - ***[RSA schemes]** using cryptographic key sizes of [**selection**: 3072, 4096, 6144, 8192]*
  - ***[ECC schemes]** using [“NIST curves” P-384 and [**selection**: P-521, no other curves]]*

  ]

] that meet the following: [**selection**: *FIPS PUB 203 [ML-KEM], FIPS PUB 204 [ML-DSA], FIPS PUB 186-5 Appendix A.1 [RSA], FIPS PUB 186-5 Appendix A.2 [ECC]*].

Application Note: Asymmetric keys may be used to “wrap” a key or submask. This SFR should be included by the ST author when making the appropriate selection in FCS_COP.1/KeyEncap.

Note that ML-DSA and ML-KEM are not usable in any functions at the time of initial publication, they are added to this requirement in support of future protocol updates. As support is expanded for CNSA 2.0, CNSA 1.0 will be removed as an selection in a future update.

FCS_CKM.6/GENHW Cryptographic Key Destruction (General Hardware)

The inclusion of this selection-based component depends upon selection in:

- ***FCS_CKM_EXT.6.1***

FCS_CKM.6.1/GENHW

The TSF shall destroy [**assignment**: *list of cryptographic keys (including keying material)*] when [*no longer needed*].

FCS_CKM.6.2/GENHW

The TSF shall destroy cryptographic keys and keying material specified in FCS_CKM.6.1/GENHW in accordance with a specified cryptographic key destruction method [**selection**:

- *For volatile memory, the destruction shall be executed by a [**selection**:*
  - *single overwrite consisting of [**selection**:*
    - *a pseudo-random pattern using the TSF's RBG,*
    - *zeroes,*
    - *ones,*
    - *a new value of a key,*
    - *[**assignment**: some value that does not contain any CSP]*

    ]
  - *removal of power to the memory,*
  - *destruction of reference to the key directly followed by a request for garbage collection*

  ]
- *For non-volatile memory, the destruction shall be executed by a [**selection**:*
  - *single*
  - *[**assignment**: ST author defined multi-pass]*

  *] overwrite consisting of [**selection**:*
  - *a pseudo-random pattern using the TSF's RBG,*
  - *zeroes,*
  - *ones,*
  - *a new value of a key of the same size,*
  - *[**assignment**: some value that does not contain any CSP]*
  - *block erase*

  ]

] that meets the following: [*no standard*].

Application Note: This SFR must be included in the ST if selected in FCS_CKM_EXT.6.

In the first selection, the ST Author is presented options for destroying disused cryptographic keys based on whether they are in volatile memory or non-volatile storage within the TOE. The selection of block erase for non-volatile storage applies only to flash memory. A block erase does not require a read-verify, since the reference to the memory location is erased as well as the data itself.

Within the selections is the option to overwrite the memory location with a new value of a key. The intent is that a new value of a key (as specified in another SFR within the cPP) can be used to "replace" an existing key.

Several selections allow assignment of a 'value that does not contain any CSP'. This means that the TOE uses some other specified data not drawn from an RBG meeting FCS_RBG.1 requirements, and not being any of the particular values listed as other selection options. The point of the phrase 'does not contain any CSP' is to ensure that the overwritten data is carefully selected, and not taken from a general 'pool' that might contain current or residual data that itself requires confidentiality protection.

Key destruction does not apply to the public component of asymmetric key pairs.

FCS_CKM.6/KEK Cryptographic Key Destruction (Key Cryptographic Erase)

The inclusion of this selection-based component depends upon selection in:

- ***FCS_CKM_EXT.6.1***

FCS_CKM.6.1/KEK

The TSF shall destroy [**assignment**: *list of cryptographic keys (including keying material)*] when [*no longer needed*].

FCS_CKM.6.2/KEK

The TSF shall destroy cryptographic keys and keying material specified by FCS_CKM.6.1/KEK in accordance with a specified cryptographic key destruction method [*by using another method in FCS_CKM_EXT.6.1 to destroy all encryption keys encrypting the key intended for destruction*] that meets the following: [*no standard*].

Application Note: A key can be considered destroyed by destroying the key that protects the key. If a key is wrapped or encrypted it is not necessary to "overwrite" that key, overwriting the key that is used to wrap or encrypt the key used to encrypt/decrypt data, using the appropriate method for the memory type involved, will suffice. For example, if a product uses a Key Encryption Key (KEK) to encrypt a Data Encryption Key (DEK), destroying the KEK using one of the methods in FCS_CKM_EXT.6.1 is sufficient, since the DEK would no longer be usable (of course, presumes the DEK is still encrypted).

FCS_CKM.6/SW Cryptographic Key Destruction (Software TOE, 3rd Party Storage)

The inclusion of this selection-based component depends upon selection in:

- ***FCS_CKM_EXT.6.1***

FCS_CKM.6.1/SW

The TSF shall destroy [**assignment**: *list of cryptographic keys (including keying material)*] when [*no longer needed*].

FCS_CKM.6.2/SW

The TSF shall destroy cryptographic keys and keying material specified in FCS_CKM.6.1/SW in accordance with a specified cryptographic key destruction method [**selection**:

- *For volatile memory, the destruction shall be executed by a [**selection**:*
  - *single overwrite consisting of [**selection**:*
    - *a pseudo-random pattern using the TSF's RBG,*
    - *zeroes,*
    - *ones,*
    - *a new value of a key,*
    - *[**assignment**: some value that does not contain any CSP]*

    ]
  - *removal of power to the memory,*
  - *destruction of reference to the key directly followed by a request for garbage collection*

]

- *For non-volatile storage that consists of the invocation of an interface provided by the underlying platform that [**selection**:*
  - *logically addresses the storage location of the key and performs a [**selection**: single, [**assignment**: ST author defined multi-pass]] overwrite consisting of [**selection**:*
    - *a pseudo-random pattern using the TSF's RBG,*
    - *zeroes,*
    - *ones,*
    - *a new value of a key of the same size,*
    - *[**assignment**: some value that does not contain any CSP]*

    ]
  - *instructs the underlying platform to destroy the abstraction that represents the key*

]

] that meets the following: [*no standard*].

Application Note: This SFR must be included in the ST if selected in FCS_CKM_EXT.6.

The interface referenced in the requirement could take different forms, the most likely of which is an application programming interface to an OS kernel. There may be various levels of abstraction visible. For instance, in a given implementation the application may have access to the file system details and may be able to logically address specific memory locations. In another implementation the application may simply have a handle to a resource and can only ask the platform to delete the resource. The level of detail to which the TOE has access will be reflected in the TSS section of the ST.

Several selections allow assignment of a 'value that does not contain any CSP'. This means that the TOE uses some other specified data not drawn from an RBG meeting FCS_RBG.1 requirements, and not being any of the particular values listed as other selection options. The point of the phrase 'does not contain any CSP' is to ensure that the overwritten data is carefully selected, and not taken from a general 'pool' that might contain current or residual data that itself requires confidentiality protection.

Key destruction does not apply to the public component of asymmetric key pairs.

FCS_CKM.6/TOEHW Cryptographic Key Destruction (TOE-Controlled Hardware)

The inclusion of this selection-based component depends upon selection in:

- ***FCS_CKM_EXT.6.1***

FCS_CKM.6.1/TOEHW

The TSF shall destroy [**assignment**: *list of cryptographic keys (including keying material)*] when [*no longer needed*].

FCS_CKM.6.2/TOEHW

The TSF shall destroy cryptographic keys and keying material specified in FCS_CKM.6.1/TOEHW in accordance with a specified cryptographic key destruction method [**selection**:

- *For volatile memory, the destruction shall be executed by a [**selection**:*
  - *single overwrite consisting of [**selection**:*
    - *a pseudo-random pattern using the TSF's RBG,*
    - *zeroes,*
    - *ones,*
    - *a new value of a key,*
    - *[**assignment**: some value that does not contain any CSP]*

    ]
  - *removal of power to the memory,*
  - *destruction of reference to the key directly followed by a request for garbage collection*

  ]
- *For non-volatile [**selection**:*
  - *that employs a wear-leveling algorithm, the destruction shall be executed by a [**selection**: single overwrite consisting of zeroes, single overwrite consisting of ones, overwrite with a new value of a key of the same size, single overwrite consisting of [**assignment**: some value that does not contain any CSP], a pseudo-random pattern using the TSF's RBG, block erase]*
  - *that does not employ a wear-leveling algorithm, the destruction shall be executed by a [**selection**:*
    - *[**selection**: single, [**assignment**: ST author defined multi-pass] overwrite consisting of zeros followed by a read-verify]*
    - *[**selection**: single, [**assignment**: ST author defined multi-pass] overwrite consisting of ones followed by a read-verify]*
    - *[**selection**: single, [**assignment**: ST author defined multi-pass]overwrite consisting of [**assignment**: some value that does not contain any CSP] followed by a read-verify]block erase*

    ]

  ] and if the read-verification of the overwritten data fails, the process shall be repeated again up to [assignment: number of times to attempt overwrite] times, whereupon an error is returned.

] that meets the following: [*no standard*].

Application Note: This SFR must be included in the ST if selected in FCS_CKM_EXT.6.

In the first selection, the ST Author is presented options for destroying a key based on the memory or storage technology where keys are stored within the TOE.

If non-volatile memory is used to store keys, the ST Author selects whether the memory storage algorithm uses wear-leveling or not. Storage technologies or memory types that use wear-leveling are not required to perform a read verify. The selection for destruction includes block erase as an option, and this option applies only to flash memory. A block erase does not require a read verify, since the mappings of logical addresses to the erased memory locations are erased as well as the data itself.

Within the selections is the option to overwrite a disused key with a new value of a key. The intent is that a new value of a key (as specified in another SFR within the cPP) can be used to "replace" an existing key.

If a selection for read verify is chosen, it should generate an audit record upon failures.

Several selections allow assignment of a ‘value that does not contain any CSP’. This means that the TOE uses some other specified data not drawn from an RBG meeting FCS_RBG.1 requirements, and not being any of the particular values listed as other selection options. The point of the phrase ‘does not contain any CSP’ is to ensure that the overwritten data is carefully selected, and not taken from a general ‘pool’ that might contain current or residual data that itself requires confidentiality protection.

Key destruction does not apply to the public component of asymmetric key pairs.

FCS_COP.1/KeyEncap Cryptographic Operation - Key Encapsulation

The inclusion of this selection-based component depends upon selection in:

- ***FCS_KYC_EXT.2.2*****,**
- ***FPT_KYP_EXT.1.1***

FCS_COP.1.1/KeyEncap

The TSF shall perform [key encapsulation] in accordance with a specified cryptographic algorithm [**selection**: *Cryptographic algorithm*] and cryptographic key sizes [**selection**: *Cryptographic key sizes*] that meet the following: [**selection**: *List of standards*]

Table 7 provides the allowed choices for completion of the selection operations of FCS_COP.1/KeyEncap.

Table 7: Allowed choices for FCS_COP.1/KeyEncap

Identifier	Cryptographic algorithm	Cryptographic key sizes	List of standards
KAS1	RSASVE	[**selection**: *3072*, *4096*, *6144*, *8192*] bits	NIST SP 800-56B Revision 2
KTS-OAEP	RSA-OAEP	[**selection**: *3072*, *4096*, *6144*, *8192*] bits	NIST SP 800-56B Revision 2
ML-KEM	ML-KEM	Parameter set = ML-KEM-1024	NIST FIPS 203

Application Note: This requirement is used in the body of the ST if the ST author chooses to use key transport in the key chaining approach that is specified in FCS_KYC_EXT.2.

NIST SP 800-57 Part 1 Revision 5 Section 5.6.2 specifies that the size of key used to protect the key being transported should be at least the security strength of the key it is protecting.

If this SFR is claimed, then FCS_CKM.1/AKG must also be claimed.

KAS1 and KTS-OAEP with the selectable parameters are CNSA 1.0 compliant. ML-KEM-1024 is CNSA 2.0 compliant.

FCS_COP.1/KeyedHash Cryptographic Operation (Keyed Hash Algorithm)

The inclusion of this selection-based component depends upon selection in:

- ***FCS_CKM.5.1*****,**

- ***FCS_RBG.1.1***,
- ***FCS_VAL_EXT.1.1***

FCS_COP.1.1/KeyedHash

The TSF shall perform [keyed hash message authentication] in accordance with a specified cryptographic algorithm [**selection**: *Keyed Hash Algorithm*] and cryptographic key sizes [**selection**: *Cryptographic key sizes*] that meet the following: [**selection**: *List of standards*]

Table 8 provides the allowed choices for completion of the selection operations of FCS_COP.1/KeyedHash.

Table 8: Allowed choices for FCS_COP.1/KeyedHash

Keyed Hash Algorithm	Cryptographic key sizes	List of standards
HMAC-SHA-256	256 bits	[**selection**: *ISO/IEC 9797-2:2021 (Section 7 "MAC Algorithm 2"), FIPS PUB 198-1*]
HMAC-SHA-384	[**selection**: *384 (ISO, FIPS), 256 (FIPS)*] bits	[**selection**: *ISO/IEC 9797-2:2021 (Section 7 "MAC Algorithm 2"), FIPS PUB 198-1*]
HMAC-SHA-512	[**selection**: *512 (ISO, FIPS), 384 (FIPS), 256 (FIPS)*] bits	[**selection**: *ISO/IEC 9797-2:2021 (Section 7 "MAC Algorithm 2"), FIPS PUB 198-1*]

Application Note: The HMAC minimum key sizes in the table are specified in ISO/IEC 9797-2:2021, which requires that the minimum key size be equal to the digest size. The FIPS standard specifies no minimum or maximum key sizes, so if FIPS PUB 198-1 is selected, larger or smaller key sizes may be used. This is indicated by the parenthesized annotations in the Cryptographic Key Sizes column.

In accordance with CNSA 1.0 and 2.0, HMAC-SHA-256 may be used only as a PRF, including as part of a key derivation function or RBG.

FCS_COP.1/KeyEnc Cryptographic Operation (Key Encryption)

The inclusion of this selection-based component depends upon selection in:

- ***FCS_KYC_EXT.2.2***,
- ***FPT_KYP_EXT.1.1***

FCS_COP.1.1/KeyEnc

The TSF shall perform [symmetric-key key encryption/decryption] in accordance with a specified cryptographic algorithm [**selection**: *Cryptographic algorithm*] and cryptographic key sizes [**selection**: *Cryptographic key sizes*] that meet the following: [**selection**: *List of standards*]

Table 9 provides the allowed choices for completion of the selection operations of FCS_COP.1/KeyEnc.

Table 9: Allowed choices for FCS_COP.1/KeyEnc

Identifier	Cryptographic algorithm	Cryptographic key sizes	List of standards
AES-CBC	AES in CBC mode with non-repeating and unpredictable IVs	256 bits	[**selection**: *ISO/IEC 18033-3:2010 (Subclause 5.2), FIPS PUB 197*] [AES] [**selection**: *ISO/IEC 10116:2017 (Clause 7), NIST SP 800-38A*] [CBC]
AES-GCM	AES in GCM mode with non-repeating IVs using [**selection**: *deterministic, RBG-based*], IV construction; the tag must be of length [**selection**: *96, 104, 112, 120, 128*] bits.	256 bits	[**selection**: *ISO/IEC 18033-3:2010 (Subclause 5.2), FIPS PUB 197*] [AES] [**selection**: *ISO/IEC 19772:2020 (Clause 10), NIST SP 800-38D*] [GCM]
AES-XTS	AES in XTS mode with unique tweak values that are consecutive non-negative integers starting at an arbitrary non-negative integer	512 bits	[**selection**: *ISO/IEC 18033-3:2010 (Subclause 5.2), FIPS PUB 197*] [AES] [**selection**: *IEEE Std. 1619-2018, NIST SP 800-38E*] [XTS]

Application Note: This SFR is required when the TSF performs key encryption as part of maintaining and deriving a key chain.

FCS_COP.1/KeyWrap Cryptographic Operation - Key Wrapping

The inclusion of this selection-based component depends upon selection in:

- ***FCS_CKM.1.1/DEK*,**
- ***FCS_KYC_EXT.2.2*,**
- ***FCS_VAL_EXT.1.1*,**
- ***FPT_KYP_EXT.1.1***

The TSF shall perform [key wrapping] in accordance with a specified cryptographic algorithm [**selection**: *Cryptographic algorithm*] and cryptographic key sizes [**selection**: *Cryptographic key sizes*] that meet the following: [**selection**: *List of standards*]

Table 10 provides the allowed choices for completion of the selection operations of FCS_COP.1/KeyWrap.

Table 10: Allowed choices for FCS_COP.1/KeyWrap

Identifier	Cryptographic algorithm	Cryptographic key sizes	List of standards
AES-KW	AES in KW mode	256 bits	[**selection**: *ISO/IEC 18033-3:2010 (Subclause 5.2), FIPS PUB 197*] [AES] [**selection**: *ISO/IEC 19772:2020 (clause 6), NIST SP 800-38F (Section 6.2)*] [KW mode]
AES-KWP	AES in KWP mode	256 bits	[**selection**: *ISO/IEC 18033-3:2010 (Subclause 5.2), FIPS PUB 197*] [AES] NIST SP 800-38F (Section 6.3) [KWP mode]
AES-CCM	AES in CCM mode with unpredictable, non-repeating nonce, minimum size of 64 bits	256 bits	[**selection**: *ISO/IEC 18033-3:2010 (Subclause 5.2), FIPS PUB 197*] [AES] [**selection**: *ISO/IEC 19772:2020 (Clause 7), NIST SP 800-38C*] [CCM]
AES-GCM	AES in GCM mode with non-repeating IVs using [**selection**: *deterministic*, *RBG-based*], IV construction; the tag must be of length [**selection**: *96*, *104*, *112*, *120*, *128*] bits.	256 bits	[**selection**: *ISO/IEC 18033-3:2010 (Subclause 5.2), FIPS PUB 197*] [AES] [**selection**: *ISO/IEC 19772:2020 (Clause 10), NIST*

SP 800-38D]
[GCM]

Application Note: This SFR is required when the TSF performs key wrapping as part of maintaining and deriving a key chain (FCS_KYC_EXT.2) or when the TSF performs validation of a submask, intermediate key, or BEV using a key wrap operation (FCS_VAL_EXT.1).

NIST 800-57p1rev5 sec. 5.6.2 specifies that the size of key used to protect the key being transported should be at least the security strength of the key it is protecting.

FCS_CKM.5 Cryptographic Key Derivation

The inclusion of this selection-based component depends upon selection in:

- ***FCS_KYC_EXT.2.2***

FCS_CKM.5.1

The TSF shall derive cryptographic keys [**selection**: *Key type*] from [**selection**: *Input parameters*] in accordance with a specified key derivation algorithm [**selection**: *Key derivation algorithm*] and specified cryptographic key sizes [**selection**: *Key sizes*] that meet the following: [**selection**: *List of standards*]

Table 11 provides the allowed choices for completion of the selection operations of FCS_CKM.5.

Table 11: Allowed choices for FCS_CKM.5

Key type	Input parameters	Key derivation algorithm	Key sizes	List of standards
KDF-CTR	[**selection**: *Direct Generation from a Random Bit Generator as specified in FCS_RBG.1, Concatenated keys*]	KPF2 - KDF in Counter Mode using [**selection**: *AES-256-CMAC, HMAC-SHA-256, HMAC-SHA-384, HMAC-SHA-512*] as the PRF	[**selection**: *256, 384, 512*] bits	[**selection**: *ISO/IEC 11770-6:2016 (Subclause 7.3.2) [KPF2], NIST SP 800-108 Revision 1 Update 1 (Section 4.1) [KDF in Counter Mode]*]

KDF-FB	[**selection**: *Direct Generation from a Random Bit Generator as specified in FCS_RBG.1, Concatenated keys*]	KPF3 - KDF in Feedback Mode using [**selection**: *AES-256-CMAC, HMAC-SHA-256, HMAC-SHA-384, HMAC-SHA-512*] as the PRF	[**selection**: *256, 384, 512*] bits	[**selection**: *ISO/IEC 11770-6:2016 (Subclause 7.3.3) [KPF3], NIST SP 800-108 Revision 1 Update 1 (Section 4.2) [KDF in Feedback Mode]*]
KDF-DPI	[**selection**: *Direct Generation from a Random Bit Generator as specified in FCS_RBG.1, Concatenated keys*]	KDF in Double Pipeline Iteration Mode using [**selection**: *AES-256-CMAC, HMAC-SHA-256, HMAC-SHA-384, HMAC-SHA-512*] as the PRF	[**selection**: *256, 384, 512*]bits	[**selection**: *ISO/IEC 11770-6:2016 (Subclause 7.3.4) [KPF4], NIST SP 800-108 Revision 1 Update 1 (Section 4.3) [KDF in Double-Pipeline Iteration Mode]*]
KDF-HASH	Shared secret	Hash function [**selection**: *SHA-384, SHA-512*]	[**selection**: *256, 384, 512*] bits	NIST SP 800-56C Revision 2 (Section 4.1, Option 1) [One-Step Key Derivation]
KDF-MAC-1S	Shared secret, salt, IV, output length, fixed information	Keyed hash [**selection**: *HMAC-SHA-256, HMAC-SHA-384, HMAC-SHA-512*]	[**selection**: *256, 384, 512*] bits	NIST SP 800-56C Revision 2 (Section 4.1, Options 2, 3) [One-Step Key Derivation]
KDF-MAC-2S	Shared secret, salt, IV, output length, fixed information, and [**selection**: *auxiliary shared secret, no other parameters*]	MAC Step [**selection**: *HMAC-SHA-256, HMAC-SHA-384, HMAC-SHA-512*] as randomness extraction and; KDF Step [**selection**: *KDF-CTR,*	[**selection**: *256, 384, 512*] bits	NIST SP 800-56C Revision 2 (Section 5) [Two-Step Key Derivation]

		KDF-FB, *KDF-DPI*].		
PBKDF2	P/password/input/BEV, S/salt, IV, kLEN/output length	PRF[**selection**: *HMAC-SHA-256*, *HMAC-SHA-384*, *HMAC-SHA-512*]	[**selection**: *256*, *384*, *512*] bits	NIST Special Publication 800-132 Recommendation for Password-Based Key Derivation

Application Note: If KDF-CTR, KDF-FB, or KDF-DPI is claimed, then either FCS_COP.1/CMAC or FCS_COP.1/KeyedHash must also be claimed, depending on the selection made for PRF.

If KDF-Hash is claimed, then FCS_COP.1/Hash must also be claimed.

If KDF-MAC-1S is claimed, then FCS_COP.1/KeyedHash must also be claimed.

If KDF-MAC-2S is claimed, then both FCS_COP.1/KeyedHash and FCS_COP.1/CMAC must also be claimed.

In KDF-MAC-2S, CMAC has been removed as a selection for the MAC step because it requires selection of 128 bits for the output key size, which is not supported in CNSA 1.0. If HMAC is selected in the MAC step, then the same HMAC is used as the KDF.

The security strengths of the Pseudo-Random functions for the key derivation methods must be sufficient for the security strength of the keys derived through those methods. Since CNSA 1.0 permits keys no smaller than 256 bits, no 128- or 192-bit PRFs are permitted. If PBKDF2 is selected a salt must be generated per FCS_SNI_EXT.1.

FCS_COP.1/CMAC Cryptographic Operation - CMAC

The inclusion of this selection-based component depends upon selection in:

- ***FCS_CKM.5.1***

FCS_COP.1.1/CMAC

The TSF shall perform [CMAC] in accordance with a specified cryptographic algorithm [**selection**: *Cryptographic algorithm*] and cryptographic key sizes [**selection**: *Cryptographic key sizes*] that meet the following: [**selection**: *List of standards*]

Table 12 provides the allowed choices for completion of the selection operations of FCS_COP.1/CMAC.

Table 12: Allowed choices for FCS_COP.1/CMAC

Identifier	Cryptographic algorithm	Cryptographic key sizes	List of standards
AES-CMAC	AES using CMAC mode	256 bits	[**selection**: *ISO/IEC 18033-3:2010 (Subclause 5.2)*, *FIPS PUB 197*] [AES] [**selection**: *: ISO/IEC 9797-1:2011 Subclause 7.6*, *NIST SP 800-38B*] [CMAC]

Application Note: The use of 256-bit keys for AES algorithms is required by CNSA 1.0 and 2.0.

FCS_RBG.1 Cryptographic Operation (Random Bit Generation)

The inclusion of this selection-based component depends upon selection in:

- ***FCS_CKM.1.1/DEK*,**
- ***FCS_CKM.6.2/GENHW*,**
- ***FCS_CKM.6.2/SW*,**
- ***FCS_CKM.6.2/TOEHW*,**
- ***FCS_COP.1.1/KeyEncap*,**
- ***FCS_COP.1.1/KeyEnc*,**
- ***FCS_COP.1.1/KeyWrap*,**
- ***FCS_CKM.5.1*,**
- ***FCS_SNI_EXT.1.1***

FCS_RBG.1.1

The TSF shall perform deterministic random bit generation services using [**selection**: *Hash_DRBG (SHA-256, SHA-384, SHA-512, SHA3-256, SHA3-384, SHA3-512), HMAC_DRBG (SHA-256, SHA-384, SHA-512, SHA3-256, SHA3-384, SHA3-512), CTR_DRBG (AES-128, AES-192, AES-256)*] in accordance with [**selection**: *ISO/IEC 18031:2011*, *NIST SP 800-90A*] after initialization with a seed.

Application Note: For Hash_DRBG and HMAC_DRBG, all allowed choices support a 256-bit security strength. For CTR_DRBG, the supported security strength is equal to the AES size. The TOE is expected to use a DRBG function that can support the security strength of the keys and random values to be generated. For example, an AES-192 CTR_DRBG can be used to generate 128-bit and 192-bit symmetric keys, but can not be used to generate 256-bit symmetric keys. More information is provided in Section 8.4 of NIST SP 800-90A.

FCS_RBG.1.2

The TSF shall use a [**selection**: *TSF noise source [**assignment**: name of noise source]*, ***multiple TSF noise sources [assignment: names of noise sources]***, *TSF interface for seeding*] for initialized seeding.

Application Note: For the selection in this requirement, the ST author selects "TSF noise source" if a single noise source is used as input to the DRBG. The ST author selects "multiple TSF noise sources" if a seed is formed from a combination of two or more noise sources within the TOE boundary. If the TSF implements two or more separate DRBGs that are seeded in separate manners, this SFR should be iterated for each DRBG. It multiple distinct noise sources exist such that each DRBG only uses one of them, then each iteration would select "TSF noise source"; "multiple TSF noise sources" is only selected if a single DRBG uses multiple noise sources for its seed. The ST author selects "TSF interface for seeding" if noise source data is generated outside the TOE boundary.

If "TSF noise source" is selected, FCS_RBG.3 must be claimed.

If "multiple TSF noise sources" is selected, FCS_RBG.4 and FCS_RBG.5 must be claimed.

If "TSF interface for seeding" is selected, FCS_RBG.2 must be claimed.

FCS_RBG.1.3

The TSF shall update the RBG state by [**selection**: *reseeding, uninstantiating and reinstantiating*] using a [**selection**: *TSF noise source [**assignment**: name of noise source], TSF interface for seeding*] in the following situations: [**selection**:

- *never*
- *on demand*
- *on the condition: [**assignment**: condition]*
- *after [**assignment**: time]*

] in accordance with [**assignment**: *list of standards*].

Application Note: This SFR is claimed when the TSF requires the use of random bit generation for submask generation (FCS_CKM.5.1) or salt generation (FCS_SNI_EXT.1).

FCS_RBG.2 Random Bit Generation (External Seeding)

The inclusion of this selection-based component depends upon selection in:

- ***FCS_RBG.1.2***

FCS_RBG.2.1

The TSF shall be able to accept a minimum input of [**assignment**: *minimum input length greater than zero*] from a TSF interface for the purpose of seeding.

Application Note: This requirement is claimed when a DRBG is seeded with entropy from one or more noise source that is outside the TOE boundary. Typically the entropy produced by an environmental noise source is conditioned such that the input length has full entropy and is therefore usable as the seed. However, if this is not the case, it should be noted what the minimum entropy rate of the noise source is so that the TSF can collect a sufficiently large sample of noise data to be conditioned into a seed value.

FCS_RBG.3 Random Bit Generation (Internal Seeding - Single Source)

The inclusion of this selection-based component depends upon selection in:

- ***FCS_RBG.1.2***

FCS_RBG.3.1

The TSF shall be able to seed the RBG using a [**selection, choose one of**: *TSF software-based noise source, TSF hardware-based noise source*] [**assignment**: *name of noise source*] with a minimum of [**assignment**: *number of bits*] bits of min-entropy.

Application Note: This requirement is claimed when a DRBG is seeded with entropy from a single noise source that is within the TOE boundary. Min-entropy should be expressed as a ratio of entropy bits to sampled bits so that the total amount of data needed to ensure full entropy is known, as well as the conditioning function by which that data is reduced in size to the seed.

FCS_RBG.4 Random Bit Generation (Internal Seeding - Multiple Sources)

The inclusion of this selection-based component depends upon selection in:

- ***FCS_RBG.1.2***

FCS_RBG.4.1

The TSF shall be able to seed the RBG using [**selection**: ***[assignment**: number] TSF software-based noise sources, [**assignment**: number] TSF hardware-based noise sources*].

Application Note: This requirement is claimed when a DRBG is seeded with entropy from multiple noise sources that are within the TOE boundary. FCS_RBG.5 defines the mechanism by which these sources are combined to ensure sufficient minimum entropy.

FCS_RBG.5 Random Bit Generation (Combining Noise Sources)

The inclusion of this selection-based component depends upon selection in:

- ***FCS_RBG.1.2***

FCS_RBG.5.1

The TSF shall [**assignment**: *combining operation*] [**selection**: *output from TSF noise sources, input from TSF interfaces for seeding)*] to create the entropy input into the derivation function as defined in [**assignment**: *list of standards*], resulting in a minimum of [**assignment**: *number of bits*] bits of min-entropy.

Application Note: Examples of typical combining operations include, but are not limited to, XORing or hashing.

FCS_SMC_EXT.1 Submask Combining

The inclusion of this selection-based component depends upon selection in:

- ***FCS_KYC_EXT.2.2*,**
- ***FPT_KYP_EXT.1.1***

FCS_SMC_EXT.1.1

The TSF shall combine submasks using the following method [**selection**: *exclusive OR (XOR), SHA-256, SHA-384, SHA-512*] to generate an [*intermediary key*].

Application Note: This requirement specifies the way that a product may combine the various submasks by using either an XOR or an approved SHA-hash. The approved hash functions are captured in FCS_COP.1/Hash.

This SFR is claimed when the TSF requires the use of submask combining as part of maintaining or deriving a key chain.

B.2 Protection of the TSF (FPT)

FPT_FLS.1 Failure with Preservation of Secure State

The inclusion of this selection-based component depends upon selection in:

- ***FCS_RBG.1.2***

FPT_FLS.1.1

The TSF shall preserve a secure state when the following types of failures occur: [*DRBG self-test failure*].

Application Note: This requirement must be included if FCS_RBG.1 is included in the ST. The intent of this requirement is to ensure that cryptographic services requiring random bit generation cannot be performed if a failure of a self-test defined in FPT_TST.1 occurs.

FPT_FUA_EXT.1 Firmware Update Authentication

The inclusion of this selection-based component depends upon selection in:

- ***FPT_TUD_EXT.1.3***

FPT_FUA_EXT.1.1

The TSF shall authenticate the source of the firmware update using the digital signature algorithm specified in FCS_COP.1/SigVer using the RTU that contains [**selection**: *the public key, hash value of the public key as specified in FCS_COP.1/Hash*]

FPT_FUA_EXT.1.2

The TSF shall only allow installation of update if the digital signature has been successfully verified as specified in FCS_COP.1/SigVer.

FPT_FUA_EXT.1.3

The TSF shall only allow modification of the existing firmware after the successful validation of the digital signature, using a mechanism as described in FPT_TUD_EXT.1.2.

Application Note: The firmware portion of the TOE (e.g., RTU (key store and the signature verification algorithm)) is expected to be stored in a write protected area on the TOE. It is expected that the firmware only be modifiable in a post-manufacturing state using the authenticated update mechanism described in FPT_FUA_EXT.1. The TSF is modifiable only by using the mechanisms specified in FPT_TUD_EXT.1.

FPT_FUA_EXT.1.4

The TSF shall return an error code if any part of the firmware update process fails.

Application Note: This SFR must be claimed if "authenticated firmware update mechanism as described in FPT_FUA_EXT.1" is claimed in FPT_TUD_EXT.1.3.

The authenticated firmware update mechanism employs digital signatures to ensure the authenticity of the firmware update image. The TSF provides a RTU that contains a signature verification algorithm and a key store that includes the public key needed to verify the signature on the update image. The key store in the RTU should include a public key used to verify the signature on an update image or a hash of the public key if a copy of the public key is provided with the update image. In the latter case, the update mechanism should hash the public key provided with the update image, and ensure that it matches a hash which appears in the key store before using the provided public key to verify the signature on the update image. If the hash of the public key is selected, the ST author may iterate the FCS_COP.1/Hash requirement - to specify the hashing functions used.

The intent of this requirement is to specify that the authenticated update mechanism should ensure that the new image has been digitally signed; and that the digital signature can be verified by using a public key before the update takes place. The requirement also specifies that the authenticated update mechanism only allows

installation of updates when the digital signature has been successfully verified by the TSF.

Appendix C - Extended Component Definitions

This appendix contains the definitions for all extended requirements specified in the PP.

C.1 Extended Components Table

All extended components specified in the PP are listed in this table:

Table 13: Extended Component Definitions

Functional Class	Functional Components
Cryptographic Support (FCS)	FCS_CKM_EXT Cryptographic Key Destruction Types FCS_KYC_EXT Key Chaining FCS_SMC_EXT Submask Combining FCS_SNI_EXT Cryptographic Operation (Salt, Nonce, and Initialization Vector Generation) FCS_VAL_EXT Validation of Cryptographic Elements
Protection of the TSF (FPT)	FPT_FAC_EXT Firmware / Software Access Control FPT_FUA_EXT Firmware Update Authentication FPT_KYP_EXT Key and Key Material Protection FPT_PWR_EXT Power Management FPT_RBP_EXT Rollback Protection FPT_TUD_EXT Trusted Update
User Data Protection	FDP_DSK_EXT Protection of Data on Disk

C.2 Extended Component Definitions

C.2.1 Cryptographic Support (FCS)

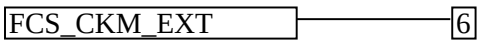
This PP defines the following extended components as part of the FCS class originally defined by CC Part 2:

C.2.1.1 FCS_CKM_EXT Cryptographic Key Destruction Types

Family Behavior

This family is intended to support the ability to specify the implementation of multiple key destruction methods.

Component Leveling



FCS_CKM_EXT.6, Cryptographic Key Destruction Types, provides the TOE with the ability to select between multiple methods of key destruction.

Management: FCS_CKM_EXT.6

There are no management functions foreseen.

Audit: FCS_CKM_EXT.6

There are no audit events foreseen.

FCS_CKM_EXT.6 Cryptographic Key Destruction Types

Hierarchical to: No other components.

Dependencies to: FCS_CKM.6 Cryptographic Key and Key Material Destruction

FCS_CKM_EXT.6.1

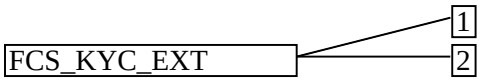
The TSF shall use [**assignment**: *one or more iterations of FCS_CKM.6 defined elsewhere in the Security Target*] key destruction methods.

C.2.1.2 FCS_KYC_EXT Key Chaining

Family Behavior

This family provides the specification to be used for using multiple layers of encryption keys to ultimately secure the protected data encrypted on the drive.

Component Leveling



FCS_KYC_EXT.1, Key Chaining (Initiator), requires the TSF to maintain a key chain for a BEV that is provided to a component external to the TOE. Note that this cPP does not include FCS_KYC_EXT.1; it is only included here to provide a complete definition of the FCS_KYC_EXT family.

FCS_KYC_EXT.2, Key Chaining (Recipient), requires the TSF to be able to accept a BEV that is then chained to a DEK used by the TSF through some method.

Management: FCS_KYC_EXT.1

There are no management functions foreseen.

Audit: FCS_KYC_EXT.1

There are no audit events foreseen.

FCS_KYC_EXT.1 Key Chaining (Initiator)

Hierarchical to: No other components.

Dependencies to: FCS_CKM.1 Cryptographic Key Generation

FCS_COP.1 Cryptographic Operation

FCS_CKM.5 Cryptographic Key Derivation

FCS_SMC_EXT.1 Submask Combining

FCS_VAL_EXT.1 Validation

FCS_KYC_EXT.1.1

The TSF shall maintain a key chain of: [**selection**:

- *one, using a submask as the BEV;*
- *intermediate keys originating from one or more submasks to the BEV using the following methods: [**selection**:*
  - *key encryption as specified in FCS_COP.1*
  - *key transport as specified in FCS_COP.1,*
  - *key wrapping as specified in FCS_COP.1,*
  - *key derivation as specified in FCS_CKM.5*
  - *key combining as specified in FCS_SMC_EXT.1,*

  ]

] while maintaining an effective strength of [**selection**: *128 bits*, *256 bits*] for symmetric keys and an effective strength of [**selection**: *not applicable*, *112 bits*, *128 bits*, *192 bits*, *256 bits*] for asymmetric keys.

FCS_KYC_EXT.1.2

The TSF shall provide at least a [**selection**: *128 bits*, *256 bits*] BEV to [**assignment**: *one or more external entities*] [**selection**:

- *after the TSF has successfully performed the validation process as specified in FCS_VAL_EXT.1*
- *without validation taking place*

]

Management: FCS_KYC_EXT.2

There are no management functions foreseen.

Audit: FCS_KYC_EXT.2

There are no audit events foreseen.

FCS_KYC_EXT.2 Key Chaining (Recipient)

Hierarchical to:	No other components.
Dependencies to:	FCS_CKM.1 Cryptographic Key Generation
	FCS_COP.1 Cryptographic Operation
	FCS_CKM.5 Cryptographic Key Derivation
	FCS_SMC_EXT.1 Submask Combining

FCS_KYC_EXT.2.1

The TSF shall accept a BEV of at least 256 bits.

FCS_KYC_EXT.2.2

The TSF shall maintain a chain of intermediary keys originating from the BEV to the DEK using the following methods: [**selection**:

- *key derivation as specified in FCS_CKM.5*
- *key wrapping as specified in FCS_COP.1*
- *key encryption as specified in FCS_COP.1*
- *key transport as specified in FCS_COP.1*
- *key combining as specified in FCS_SMC_EXT.1*

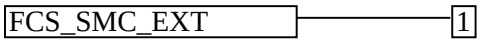
] while maintaining an effective strength of [256 bits] for symmetric keys and an effective strength of [**selection**: *not applicable, 128 bits, 192 bits, 256 bits*] for asymmetric keys.

C.2.1.3 FCS_SMC_EXT Submask Combining

Family Behavior

This family specifies the means by which submasks are combined, if the TOE supports more than one submask being used to derive or protect the BEV.

Component Leveling



FCS_SMC_EXT.1, Submask Combining, requires the TSF to combine the submasks in a predictable fashion.

Management: FCS_SMC_EXT.1

There are no management functions foreseen.

Audit: FCS_SMC_EXT.1

There are no audit events foreseen.

FCS_SMC_EXT.1 Submask Combining

Hierarchical to: No other components.

Dependencies to: FCS_COP.1 Cryptographic Operation

FCS_SMC_EXT.1.1

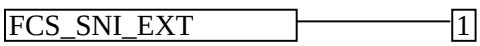
The TSF shall combine submasks using the following method [**selection**: *exclusive OR (XOR), SHA-256, SHA-384, SHA-512*] to generate an [**assignment**: *types of keys*].

C.2.1.4 FCS_SNI_EXT Cryptographic Operation (Salt, Nonce, and Initialization Vector Generation)

Family Behavior

This family ensures that salts, nonces, and IVs are well formed.

Component Leveling



FCS_SNI_EXT.1, Cryptographic Operation (Salt, Nonce, and Initialization Vector Generation), requires the generation of salts, nonces, and IVs to be used by the cryptographic components of the TOE to be performed in the specified manner.

Management: FCS_SNI_EXT.1

There are no management functions foreseen.

Audit: FCS_SNI_EXT.1

There are no audit events foreseen.

FCS_SNI_EXT.1 Cryptographic Operation (Salt, Nonce, and Initialization Vector Generation)

Hierarchical to: No other components.

Dependencies to: FCS_RBG.1 Cryptographic Operation (Random Bit Generation)

FCS_SNI_EXT.1.1

The TSF shall [**selection**:

- *use no salts*
- *use salts that are generated by a [**selection**: DRBG as specified in FCS_RBG.1, DRBG provided by the OE]*

].

FCS_SNI_EXT.1.2

The TSF shall use [**selection**: *no nonces, unique nonces with a minimum size of [**assignment**: number of bits] bits*].

FCS_SNI_EXT.1.3

The TSF shall [**selection**:

- *use no IVs*
- *create IVs in the following manner [**selection**:*
  - *CBC: IVs shall be non-repeating and unpredictable*
  - *CCM: Nonce shall be non-repeating and unpredictable*
  - *XTS: No IV. Tweak values shall be non-negative integers, assigned consecutively, and starting at an arbitrary non-negative integer;*
  - *GCM: IV shall be non-repeating. The number of invocations of GCM shall not exceed 2^32 for a given secret key*

  ]

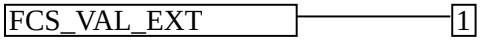
].

C.2.1.5 FCS_VAL_EXT Validation of Cryptographic Elements

Family Behavior

This family specifies the means by which submasks and/or BEVs are determined to be valid prior to their use.

Component Leveling



FCS_VAL_EXT.1, Validation, requires the TSF to validate submasks and BEVs by one or more of the specified methods.

Management: FCS_VAL_EXT.1

There are no management functions foreseen.

Audit: FCS_VAL_EXT.1

There are no audit events foreseen.

FCS_VAL_EXT.1 Validation

Hierarchical to:	No other components.
Dependencies to:	FCS_COP.1 Cryptographic Operation

FCS_VAL_EXT.1.1

The TSF shall perform validation of the [**selection**: *submask, intermediate key, BEV*] using the following methods: [**selection**:

- *key wrap as specified in FCS_COP.1;*
- *hash the [**selection**: submask, intermediate key, BEV] as specified in [**assignment**: cryptographic operation requirement] and compare it to a stored hashed [**selection**: submask, intermediate key, BEV];*
- *decrypt a known value using the [**selection**: submask, intermediate key, BEV] specified in FCS_COP.1 and compare it against a stored known value*

].

FCS_VAL_EXT.1.2

The TSF shall require validation of the [**selection**: *submask, intermediate key, BEV*] prior to [**assignment**: *activity requiring validation*].

FCS_VAL_EXT.1.3

The TSF shall [**selection**:

- *perform a key sanitization of the DEK upon a [**selection**: configurable number, [**assignment**: ST author specified number]] of consecutive failed validation attempts*
- *institute a delay such that only [**assignment**: ST author specified number of attempts] can be made within a 24 hour period*
- *block validation after [**assignment**: ST author specified number of attempts] of consecutive failed validation attempts*
- *require power cycle or reset of the TOE after [**assignment**: ST author specified number of attempts] of consecutive failed validation attempts*

].

C.2.2 Protection of the TSF (FPT)

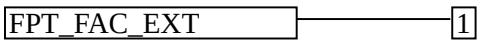
This PP defines the following extended components as part of the FPT class originally defined by CC Part 2:

C.2.2.1 FPT_FAC_EXT Firmware / Software Access Control

Family Behavior

This family requires that a valid authentication factor be provided prior to the TSF authorizing an update.

Component Leveling



FPT_FAC_EXT.1, Firmware / Software Access Control, requires the TSF to require an authentication factor or action prior to allowing an update to be performed.

Management: FPT_FAC_EXT.1

The following actions could be considered for the management functions in FMT:

- management of the password used to authorize the update

Audit: FPT_FAC_EXT.1

There are no audit events foreseen.

FPT_FAC_EXT.1 Firmware / Software Access Control

Hierarchical to: No other components.

Dependencies to: No dependencies.

FPT_FAC_EXT.1.1

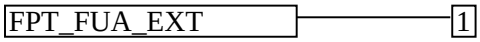
The TSF shall require [**selection**: *a password*, *a unique value printed on the device*, *a authorized user action*] before the [**selection**: *software*, *firmware*] update proceeds.

C.2.2.2 FPT_FUA_EXT Firmware Update Authentication

Family Behavior

This family requires that firmware updates be authenticated by the TSF prior to being applied.

Component Leveling



FPT_FUA_EXT.1, Firmware Update Authentication, requires the TSF to authenticate firmware updates using a specified method.

Management: FPT_FUA_EXT.1

There are no management functions foreseen.

Audit: FPT_FUA_EXT.1

There are no audit events foreseen.

FPT_FUA_EXT.1 Firmware Update Authentication

Hierarchical to: No other components.

Dependencies to: FCS_COP.1 Cryptographic Operation

FPT_FUA_EXT.1.1

The TSF shall authenticate the source of the firmware update using the digital signature algorithm specified in FCS_COP.1 using the RTU that contains [**selection**: *the public key*, *hash value of the public key as specified in FCS_COP.1*]

FPT_FUA_EXT.1.2

The TSF shall only allow installation of update if the digital signature has been successfully verified as specified in FCS_COP.1.

FPT_FUA_EXT.1.3

The TSF shall only allow modification of the existing firmware after the successful validation of the digital signature, using a mechanism as described in FPT_TUD_EXT.1.2.

FPT_FUA_EXT.1.4

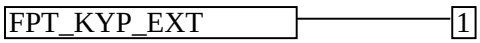
The TSF shall return an error code if any part of the firmware update process fails.

C.2.2.3 FPT_KYP_EXT Key and Key Material Protection

Family Behavior

This family requires that key and key material be protected if and when written to non-volatile storage.

Component Leveling



FPT_KYP_EXT.1, Protection of Key and Key Material, requires the TSF to ensure that no plaintext key or key material are written to non-volatile storage.

Management: FPT_KYP_EXT.1

There are no management functions foreseen.

Audit: FPT_KYP_EXT.1

There are no audit events foreseen.

FPT_KYP_EXT.1 Protection of Key and Key Material

Hierarchical to:	No other components.
Dependencies to:	FCS_COP.1 Cryptographic Operation
	FCS_KYC_EXT.1 Key Chaining (Initiator)
	FCS_KYC_EXT.2 Key Chaining (Recipient)
	FCS_SMC_EXT.1 Submask Combining

FPT_KYP_EXT.1.1

The TSF shall [**selection**:

- *not store keys in non-volatile memory*

- *only store keys in non-volatile memory when wrapped, as specified in FCS_COP.1, or encrypted, as specified in FCS_COP.1*
- *only store plaintext keys that meet any one of the following criteria [**selection**:*
  - *the plaintext key is not part of the key chain as specified in FCS_KYC_EXT.2*
  - *the plaintext key will no longer provide access to the encrypted data after initial provisioning*
  - *the plaintext key is a key split that is combined as specified in FCS_SMC_EXT.1, and the other half of the key split is [**selection**:*
    - *wrapped as specified in FCS_COP.1*
    - *encrypted as specified in FCS_COP.1*
    - *derived and not stored in non-volatile memory*

    ]
  - *the non-volatile memory the key is stored on is located in an external storage device for use as an authorization factor*
  - *the plaintext key is only used to provide additional cryptographic protection to other keys, such that disclosure of the plaintext key would not compromise the security of the keys being protected*

]

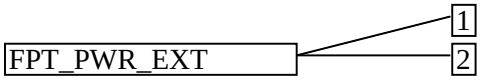
].

C.2.2.4 FPT_PWR_EXT Power Management

Family Behavior

This family defines secure behavior of the TSF when the TOE supports multiple power saving states. The use of compliant power saving states (i.e. power saving states that purge security relevant data upon entry) is essential for ensuring that state transitions cannot be used as attack vectors to bypass TOE self-protection mechanisms.

Component Leveling



FPT_PWR_EXT.1, Power Saving States, defines the compliant power saving states that are implemented by the TSF.

FPT_PWR_EXT.2, Timing of Power Saving States, describes the situations that cause compliant power saving states to be entered.

Management: FPT_PWR_EXT.1

The following actions could be considered for the management functions in FMT:

- Enable or disable the use of individual power saving states
- Specify one or more power saving state configurations

Audit: FPT_PWR_EXT.1

There are no auditable events foreseen.

FPT_PWR_EXT.1 Power Saving States

Hierarchical to: No other components.

Dependencies to: No dependencies

FPT_PWR_EXT.1.1

The TSF shall define the following compliant power saving states: [**selection**: *S3*, *S4*, *G2(S5)*, *G3*, *D0*, *D1*, *D2*, *D3*, *[**assignment**: other power saving states]*].

Management: FPT_PWR_EXT.2

There are no management functions foreseen.

Audit: FPT_PWR_EXT.2

The following actions should be auditable if FAU_GEN Security audit data generation is included in the cPP/ST:

- Transition of the TSF into different power saving states

FPT_PWR_EXT.2 Timing of Power Saving States

Hierarchical to: No other components.

Dependencies to: FPT_PWR_EXT.1 Power Saving States

FPT_PWR_EXT.2.1

For each compliant power saving state defined in FPT_PWR_EXT.1.1, the TSF shall enter the compliant power saving state when the following conditions occur: user-initiated request, [**selection**: *shutdown*, *user inactivity*, *request initiated by remote management system*, *[**assignment**: other conditions]*, *no other conditions*].

C.2.2.5 FPT_RBP_EXT Rollback Protection

Family Behavior

This family requires that the TSF protects against rollbacks or downgrades to its version.

Component Leveling

FPT_RBP_EXT ——— 1

FPT_RBP_EXT.1, Rollback Protection, requires the TSF to detect and prevent unauthorized rollback.

Management: FPT_RBP_EXT.1

There are no management functions foreseen.

Audit: FPT_RBP_EXT.1

There are no audit events foreseen.

FPT_RBP_EXT.1 Rollback Protection

Hierarchical to: No other components.

Dependencies to: No dependencies.

FPT_RBP_EXT.1.1

The TSF shall verify that the new update is not downgrading to a lower security version number by [**assignment**: *method of verifying the security version number is the same as or higher than the currently installed version*].

FPT_RBP_EXT.1.2

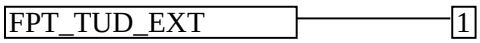
The TSF shall generate and return an error code if the attempted update package is detected to be an invalid version.

C.2.2.6 FPT_TUD_EXT Trusted Update

Family Behavior

Components in this family address the requirements for updating the TOE firmware / software.

Component Leveling



FPT_TUD_EXT.1, Trusted Update, requires the capability to be provided to update the TOE firmware and software, including the ability to verify the updates prior to installation.

Management: FPT_TUD_EXT.1

The following actions could be considered for the management functions in FMT:

- Ability to update the TOE and to verify the updates

Audit: FPT_TUD_EXT.1

The following actions should be auditable if FAU_GEN Security audit data generation is included in the cPP/ST:

- Initiation of the update process
- Any failure to verify the integrity of the update

FPT_TUD_EXT.1 Trusted Update

Hierarchical to: No other components.

Dependencies to: FCS_COP.1 Cryptographic Operation

FPT_TUD_EXT.1.1

The TSF shall provide [**assignment**: *list of subjects*] the ability to query the current version of the TOE [**selection**: *software*, *firmware*].

FPT_TUD_EXT.1.2

The TSF shall provide [**assignment**: *list of subjects*] the ability to initiate updates to TOE [**selection**: *software*, *firmware*].

FPT_TUD_EXT.1.3

The TSF shall verify updates to the TOE software using a [**selection**: *digital signature*, *published hash*] by the manufacturer prior to installing those updates.

C.2.3 User Data Protection

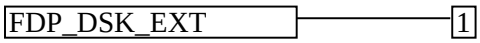
This PP defines the following extended components as part of the class originally defined by CC Part 2:

C.2.3.1 FDP_DSK_EXT Protection of Data on Disk

Family Behavior

This family specifies methods for ensuring that data residing in permanent storage on disk is not subject to unauthorized disclosure.

Component Leveling



FDP_DSK_EXT.1, Protection of Data on Disk, requires the TSF to encrypt all protected data without user intervention using full drive encryption.

Management: FDP_DSK_EXT.1

There are no management functions foreseen.

Audit: FDP_DSK_EXT.1

There are no audit events foreseen.

FDP_DSK_EXT.1 Protection of Data on Disk

Hierarchical to: No other components.

Dependencies to: FCS_COP.1 Cryptographic Operation

FDP_DSK_EXT.1.1

The TSF shall perform Full Drive Encryption in accordance with FCS_COP.1, such that the drive contains no plaintext protected data.

FDP_DSK_EXT.1.2

The TSF shall encrypt all protected data without user intervention.

Appendix D - Entropy Documentation and Assessment

This is an optional appendix in the cPP, and only applies if the TOE is providing deterministic random bit generation services, e.g. the ST claims FCS_RBG.1.

This appendix describes the required supplementary information for each entropy source used by the TOE.

The documentation of the entropy sources should be detailed enough that, after reading, the evaluator will thoroughly understand the entropy source and why it can be relied upon to provide sufficient entropy. This documentation should include multiple detailed sections: design description, entropy justification, operating conditions, and health testing. This documentation is not required to be part of the TSS in the public facing ST.

D.1 Design Description

Documentation shall include the design of each entropy source as a whole, including the interaction of all entropy source components. Any information that can be shared regarding the design should also be included for any third-party entropy sources that are included in the product.

The documentation will describe the operation of the entropy source to include how entropy is produced, and how unprocessed (raw) data can be obtained from within the entropy source for testing purposes. The documentation should walk through the entropy source design indicating where the entropy comes from, where the entropy output is passed next, any post-processing of the raw outputs (hash, XOR, etc.), if/where it is stored, and finally, how it is output from the entropy source. Any conditions placed on the process (e.g., blocking) should also be described in the entropy source design. Diagrams and examples are encouraged.

This design must also include a description of the content of the security boundary of the entropy source and a description of how the security boundary ensures that an adversary outside the boundary cannot affect the entropy rate.

If implemented, the design description shall include a description of how third-party applications can add entropy to the RBG. A description of any RBG state saving between power-off and power-on shall be included.

D.2 Entropy Justification

There should be a technical argument for where the unpredictability in the source comes from and why there is confidence in the entropy source delivering sufficient entropy for the uses made of the RBG output (by this particular TOE). This argument will include a description of the expected min-entropy rate (i.e. the minimum entropy (in bits) per bit or byte of source data) and explain that sufficient entropy is going into the TOE randomizer seeding process. This discussion will be part of a justification for why the entropy source can be relied upon to produce bits with entropy.

The amount of information necessary to justify the expected min-entropy rate depends on the type of entropy source included in the product.

For developer provided entropy sources, in order to justify the min-entropy rate, it is expected that a large number of raw source bits will be collected, statistical tests will be performed, and the min-entropy rate determined from the statistical tests. While no particular statistical tests are required at this time, it is expected that some testing is necessary in order to determine the amount of min-entropy in each output.

For third party provided entropy sources, in which the TOE vendor has limited access to the design and raw entropy data of the source, the documentation will indicate an estimate of the amount of min-entropy obtained from this third-party source. It is acceptable for the vendor to “assume” an amount of min-entropy, however, this assumption must be clearly stated in the documentation provided. In particular, the min-entropy estimate must be specified and the assumption included in the ST.

Regardless of type of entropy source, the justification will also include how the DRBG is initialized with the entropy stated in the ST, for example by verifying that the min-entropy rate is multiplied by the amount of source data used to seed the DRBG or that the rate of entropy expected based on the amount of source data is explicitly stated and compared to the statistical rate. If the amount of source data used to seed the DRBG is not clear or the calculated rate is not explicitly related to the seed, the documentation will not be considered complete.

The entropy justification shall not include any data added from any third-party application or from any state saving between restarts.

D.3 Operating Conditions

The entropy rate may be affected by conditions outside the control of the entropy source itself. For example, voltage, frequency, temperature, and elapsed time after power-on are just a few of the factors that may affect the operation of the entropy source. As such, documentation will also include the range of operating conditions under which the entropy source is expected to generate random data. Similarly, documentation shall describe the conditions under which the entropy source is no longer guaranteed to provide sufficient entropy. Methods used to detect failure or degradation of the source shall be included.

D.4 Health Testing

More specifically, all entropy source health tests and their rationale will be documented. This will include a description of the health tests, the rate and conditions under which each health test is performed (e.g., at startup, continuously, or on-demand), the expected results for each health test, TOE behavior upon entropy source failure, and rationale indicating why each test is believed to be appropriate for detecting one or more failures in the entropy source.

Appendix E - Key Management Description

The documentation of the product's encryption key management should be detailed enough that, after reading, the evaluator will thoroughly understand the product's key management and how it meets the requirements to ensure the keys are adequately protected. This documentation should include an essay and diagrams. This documentation is not required to be part of the TSS - it can be submitted as a separate document and marked as developer proprietary.

Essay:

The essay will provide the following information for all keys in the key chain:

- The purpose of the key
- If the key is stored in non-volatile memory
- How and when the key is protected
- How and when the key is derived
- The strength of the key
- When or if the key would be no longer needed, along with a justification.

The essay will also describe the following topics:

- A description of all authorization factors that are supported by the product and how each factor is handled, including any conditioning and combining performed.
- If validation is supported, the process for validation shall be described, noting what value is used for validation and the process used to perform the validation. It shall describe how this process ensures no keys in the key chain are weakened or exposed by this process.
- The authorization process that leads to the ultimate release of the BEV. This section shall detail the key chain used by the product. It shall describe which keys are used in the protection of the BEV and how they meet the derivation, key wrap, or a combination of the two requirements, including the direct chain from the initial authorization to the BEV. It shall also include any values that add into that key chain or interact with the key chain and the protections that ensure those values do not weaken or expose the overall strength of the key chain.
- The diagram and essay will clearly illustrate the key hierarchy to ensure that at no point the chain could be broken without a cryptographic exhaust or all of the initial authorization values and the effective strength of the BEV is maintained throughout the Key Chain.
- A description of the data encryption engine, its components, and details about its implementation (e.g. for hardware: integrated within the device's main SOC or separate co-processor, for software: initialization of the product, drivers, libraries (if applicable), logical interfaces for encryption/decryption, and areas which are not encrypted (e.g. boot loaders, portions associated with the Master Boot Record (MBRs), partition tables, etc.)). The description should also include the data flow from the device's host interface to the device's persistent media storing the data, information on those conditions in which the data bypasses the data encryption engine (e.g. read-write operations to an unencrypted Master Boot Record area). The description should be detailed enough to verify all platforms to ensure that when the user enables encryption, the product encrypts all hard storage devices. It should also describe the platform's boot initialization, the encryption initialization process, and at what moment the product enables the encryption.
- The process for destroying keys when they are no longer needed by describing the storage location of all keys and the protection of all keys stored in non-volatile memory.

Diagram:

- The diagram will include all keys from the initial authorization factors to the BEV and any keys or values that contribute into the chain. It must list the cryptographic strength of each key and indicate how each key along the chain is protected with either key derivation or key wrapping (from the allowed options). The diagram should indicate the input used to derive or unwrap each key in the chain.
- A functional (block) diagram showing the main components (such as memories and processors) and the data path between, for hardware, the device's host interface and the device's persistent media storing the data, or for

software, the initial steps needed for the activities the TOE performs to ensure it encrypts the storage device entirely when a user or administrator first provisions the product. The hardware encryption diagram shall show the location of the data encryption engine within the data path.

- The hardware encryption diagram shall show the location of the data encryption engine within the data path. The evaluator shall validate that the hardware encryption diagram contains enough detail showing the main components within the data path and that it clearly identifies the data encryption engine.

Appendix F - Acronyms

Table 14: Acronyms

Acronym	Meaning
AA	Authorization Acquisition
AES	Advanced Encryption Standard
Base-PP	Base Protection Profile
BEV	Border Encryption Value
BIOS	Basic Input Output System
CBC	Cipher Block Chaining
CC	Common Criteria
CCM	Counter with CBC-Message Authentication Code
CEM	Common Evaluation Methodology
cPP	Collaborative Protection Profile
DEK	Data Encryption Key
DRBG	Deterministic Random Bit Generator
DSS	Digital Signature Standard
ECC	Elliptic Curve Cryptography
ECDSA	Elliptic Curve Digital Signature Algorithm
EE	Encryption Engine
EEPROM	Electrically Erasable Programmable Read-Only Memory
EP	Extended Package
FDE	Full Drive Encryption
FFC	Finite Field Cryptography
FIPS	Federal Information Processing Standards
FP	Functional Package
GCM	Galois Counter Mode
HMAC	Keyed-Hash Message Authentication Code

HW	Hardware
IEEE	Institute of Electrical and Electronics Engineers
ISO/IEC	International Organization for Standardization / International Electrotechnical Commission
IT	Information Technology
ITSEF	IT Security Evaluation Facility
IV	Initialization Vector
KEK	Key Encryption Key
KMD	Key Management Description
KRK	Key Release Key
MBR	Master Boot Record
NIST	National Institute of Standards and Technology
OE	Operational Environment
OS	Operating System
PBKDF	Password-Based Key Derivation Function
PP	Protection Profile
PP-Configuration	Protection Profile Configuration
PP-Module	Protection Profile Module
PRF	Pseudo Random Function
RBG	Random Bit Generator
RNG	Random Number Generator
RoT	Root of Trust
RSA	Rivest Shamir Adleman Algorithm
RTU	Root of Trust for Update
SAR	Security Assurance Requirement
SED	Self-Encrypting Drive
SFR	Security Functional Requirement
SHA	Secure Hash Algorithm
SPD	Security Problem Definition
SPI	Serial Peripheral Interface

ST	Security Target
TOE	Target of Evaluation
TPM	Trusted Platform Module
TSF	TOE Security Functionality
TSF	TOE Security Functionality
TSFI	TSF Interface
TSS	TOE Summary Specification
TSS	TOE Summary Specification
USB	Universal Serial Bus
XOR	Exclusive or
XTS	XEX (XOR Encrypt XOR) Tweakable Block Cipher with Ciphertext Stealing

Appendix G - Bibliography

Table 15: Bibliography

Identifier	Title
[CC]	Common Criteria for Information Technology Security Evaluation - • Part 1: Introduction and general model, CCMB-2022-11-001, CC:2022, Revision 1, November 2022. • Part 2: Security functional requirements, CCMB-2022-11-002, CC:2022, Revision 1, November 2022. • Part 3: Security assurance requirements, CCMB-2022-11-003, CC:2022, Revision 1, November 2022. • Part 4: Framework for the specification of evaluation methods and activities, CCMB-2022-11-004, CC:2022, Revision 1, November 2022. • Part 5: Pre-defined packages of security requirements, CCMB-2022-11-005, CC:2022, Revision 1, November 2022.
[CEM]	Common Methodology for Information Technology Security Evaluation - • Evaluation methodology, CCMB-2022-11-006, CC:2022, Revision 1, November 2022.
[FDE–EE]	collaborative Protection Profile for Full Drive Encryption – Encryption Engine, Version 3.0, March, 2026